

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN

MÔN LẬP TRÌNH KỊCH BẢN TỰ ĐỘNG HÓA CHO QUẢN TRỊ VÀ BẢO MẬT MẠNG

Đề tài:

CIS GOOGLE KUBERNETES ENGINE (GKE) AUTOPILOT BENCHMARK

Giảng viên hướng dẫn: Th.S. Trần Thị Dung, ThS. Văn Thiên Luân

Lớp: NT542.Q11

Nhóm thực hiện: Nhóm 20

STT	Họ và tên	MSSV
1	Đinh Huỳnh Gia Bảo	22520101
2	Nguyễn Xuân Cường	22520178

THÀNH PHỐ HỒ CHÍ MINH, 2025

MỤC LỤC

MỤC LỤC.....	2
DANH MỤC HÌNH ẢNH.....	4
DANH MỤC VIẾT TẮT	6
LỜI CẢM ƠN	8
1. TỔNG QUAN.....	9
1.1.Bối cảnh thực tế.....	9
1.2.Giới thiệu	10
1.2.1. Tổng quan về Kubernetes.....	10
1.2.1.1 Định nghĩa	10
1.2.1.2 Kiến trúc Kubernetes	10
1.2.2. Tổng quan về Google Kubernetes Engine (GKE).....	12
1.2.2.1 Định nghĩa GKE	12
1.2.2.2 So sánh Standard và Autopilot	12
1.2.3 Giới thiệu về CIS và CIS Benchmark.....	13
1.2.4 Giới thiệu CIS Google Kubernetes Engine Autopilot Benchmark.....	13
1.3.Nội dung Benchmark	14
1.3.1 Policies	14
1.3.2 Managed Services.....	16
2. PHƯƠNG PHÁP.....	17
2.1 Manual	17
2.2 Automation	17
3. TRIỂN KHAI.....	18
3.1. Mô hình.....	18
3.1.1. Mô hình kiến trúc	18
3.1.2 Sơ đồ luồng làm việc.....	20
3.2. Khởi tạo cụm	21
3.2.1 Tạo cụm GKE Autopilot	21
3.2.1.1 Phương pháp Manual (tạo bằng UI)	21

3.2.1.2 Phương pháp Automation	27
3.2.2 Tạo cụm GKE Standard.....	29
3.2.2.1 Phương pháp Manual (tạo bằng UI)	29
3.2.2.2 Phương pháp Automation.....	38
3.3. Triển khai Manual	40
3.3.1: Đảm bảo vai trò cluster-admin chỉ được sử dụng khi cần thiết.....	40
3.3.2: Giảm thiểu việc sử dụng ký tự đại diện (wildcard) trong Roles và ClusterRoles	41
3.3.3: Giới hạn việc sử dụng quyền Bind, Impersonate và Escalate trong Kubernetes cluster	42
3.3.4: Đảm bảo cụm áp dụng và thực thi hồ sơ Pod Security Standard Baseline cho tất cả các namespace.....	43
3.3.5: Tạo ranh giới quản trị giữa các tài nguyên bằng cách sử dụng namespaces	44
3.3.6 Bật VPC Flow Logs và khả năng quan sát lưu lượng nội bộ giữa các node	45
3.4 Triển khai Automation	46
3.4.1 Kiểm tra quyền non-admin clusterrole-binding	46
3.4.2. Pod Security Standard (PSS):.....	47
3.4.3 General Policy: Ranh giới giữa namespace.....	47
3.4.4 Cluster Networking: Bật VPC flowlogs và IntraNodeVisibility	48
4. ĐÁNH GIÁ VÀ KẾT LUẬN	50
4.1 . Bảng so sánh.....	50
4.2 Tổng kết đồ án.....	50
PHỤ LỤC.....	52
NHẬN XÉT CỦA GIẢNG VIÊN	53
TÀI LIỆU THAM KHẢO.....	54

DANH MỤC HÌNH ẢNH

Hình 1. Logo Kubernetes	10
Hình 2. Mô hình kiến trúc K8s.....	11
Hình 3. Logo GKE	12
Hình 4. Logo của CIS.....	13
Hình 5. CIS Google Kubernetes Engine (GKE) Autopilot Benchmark v1.2.0	14
Hình 6. Mô hình kiến trúc	18
Hình 7. Sơ đồ luồng làm việc.....	20
Hình 8. IAM & Admin.....	22
Hình 9. Nhập role cần thiết	22
Hình 10. Chọn Kubernetes Engine.....	24
Hình 11. Giao diện tổng quát cụm Kubernetes	24
Hình 12. Chọn Autopilot.....	25
Hình 13. Nhập thông tin cụm Kubernetes Autopilot.....	25
Hình 14. Hoàn tất tạo cụm Kubernetes Autopilot.....	26
Hình 15. Thông tin tổng quát cụm Autopilot	27
Hình 16. Cấu trúc thư mục Kubernetes Autopilot.....	28
Hình 17. Chọn chế độ cấu hình Standard.....	29
Hình 18. Điền tên cụm	30
Hình 19. Cấu hình Location Type.....	31
Hình 20. Chọn phiên bản Kubernetes Standard	31
Hình 21. Cấu hình Node pool.....	32
Hình 22. Cấu hình Image type	33
Hình 23. Cấu hình phân cứng.....	33
Hình 24. Cấu hình Control Plane Access	34
Hình 25. Cấu hình VPC	35
Hình 26. Cấu hình IP Management	36
Hình 27. Cấu hình tùy chọn Security	37
Hình 28. Kết quả khi tạo cụm GKE Standard	38
Hình 29. Kết quả workflow.....	38
Hình 29. Cấu trúc mã nguồn Terraform tạo Kubernetes Standard.....	39

Hình 30. Kiểm tra vai trò cluster-admin	40
Hình 31. Autopilot mặc định chặn non-admin user	41
Hình 32. Kiểm tra quyền wildcard.....	41
Hình 33. Autopilot chặn wildcard không phù hợp.....	42
Hình 34. Kiểm tra quyền Bind, Impersonate và Escalate	42
Hình 35. Tạo quyền độc hại	42
Hình 36. Xóa quyền vừa tạo.....	42
Hình 37. Autopilot tự động chặn.....	43
Hình 38. Autopilot chặn tạo quyền escalate và impersonate	43
Hình 39. Kiểm tra namespace nào chưa gán PSS	43
Hình 40. Áp dụng PSS	43
Hình 41. Autopilot bật sẵn PSS.....	44
Hình 42. Kiểm tra phân tách namespace.....	44
Hình 43. Tạo namespace cho dev và áp dụng	44
Hình 44. Autopilot tự động giới hạn quyền namespace	45
Hình 45. Autopilot xóa quyền namespace không phù hợp.....	45
Hình 46. Kiểm tra VPC flow logs	45
Hình 47. Kiểm tra Intranode Visibility	45
Hình 48. Autopilot tự động bật VPC Flow Logs	45
Hình 48. Autopilot tự động bật Intranode Visibility	45
Hình 49. Bash script cho Kiểm tra quyền non-admin	46
Hình 50. Bash script cho PSS.....	47
Hình 51. Bash script cho namespace.....	47
Hình 52. Bash script cho VPC Flow logs.....	48
Hình 53. Bash script cho IntraNodeVisibility	48
Hình 54. Chạy script tổng hợp	49
Hình 55. File log.txt	49
Hình 56. File log bằng HTML	50

DANH MỤC VIẾT TẮT

Số thứ tự	Chữ viết tắt	Ý nghĩa	Mô tả
1	GKE	Google Kubernetes Engine	Một dịch vụ quản lý Kubernetes của Google Cloud, mang đến một giải pháp cho việc triển khai, cân bằng tải và quản lý ứng dụng dựa trên container.
2	CIS	Center for Internet Security	Một tổ chức phi lợi nhuận ở Mỹ, chuyên tập trung vào việc nâng cao mức độ an toàn thông tin và khả năng phòng thủ mạng.
3	CLI	Command Line Interface	Phương pháp người dùng tương tác với hệ thống thông qua các lệnh dạng văn bản thay vì dùng giao diện đồ họa.
4	SDK	Software Development Kit	Gói công cụ phát triển phần mềm, bao gồm thư viện mã nguồn, trình biên dịch, trình gỡ lỗi, tài liệu, mẫu mã, và hướng dẫn
5	OS	Operating System	Là phần mềm hệ thống quản lý tài nguyên phần cứng và phần mềm của máy tính và cung cấp các dịch vụ chung cho các chương trình ứng dụng.
6	RBAC	Role-Based Access Control	Mô hình điều khiển truy cập trong đó quyền được gán cho các vai trò
7	KMS	Key Management Service	Dịch vụ tập trung quản lý vòng đời các khóa mã hóa trong hệ thống
8	IAM	Identity and Access Management	Tập hợp quy trình, công nghệ và chính sách để xác thực và ủy quyền
9	K8s	Kubernetes	Tên viết tắt phổ biến của Kubernetes, hệ thống mã nguồn mở để quản lý container.
10	GCP	Google Cloud Platform	Nền tảng điện toán đám mây do Google cung cấp.
11	VPC	Virtual Private Cloud	Mạng riêng ảo, cung cấp khả năng quản lý mạng biệt lập trên đám mây.
12	PSS	Pod Security Standards	Các tiêu chuẩn bảo mật nhằm kiểm soát cấu hình an toàn cho Pod.
13	VM	Virtual Machine	Máy ảo, đơn vị tính toán cơ bản chạy trên các node.

14	CI/CD	Continuous Integration / Continuous Deployment	Quy trình tự động hóa tích hợp và triển khai phần mềm liên tục (sử dụng Github Actions trong đồ án).
15	API	Application Programming Interface	Giao diện lập trình ứng dụng, ví dụ như Kubernetes API Server.
16	CNI	Container Network Interface	Chuẩn giao diện mạng cho container.
17	IPAM	IP Address Management	Quản lý địa chỉ IP tự động cho cluster.
18	CIDR	Classless Inter-Domain Routing	Phương pháp phân bổ địa chỉ IP và định tuyến (Dải địa chỉ IP).
19	GCS	Google Cloud Storage	Dịch vụ lưu trữ đối tượng của Google Cloud (dùng để lưu file state Terraform).
20	DNS	Domain Name System	Hệ thống phân giải tên miền, dùng để truy cập Control Plane.

LỜI CẢM ƠN

Để hoàn thành được đồ án môn Lập trình kịch bản tự động hóa cho quản trị và bảo mật mạng này, Nhóm 20 chúng em xin gửi lời cảm ơn chân thành về sự chỉ dạy, hướng dẫn tận tình của cô ThS. Trần Thị Dung và thầy ThS. Văn Thiên Luân, giảng viên Khoa Mạng máy tính và Truyền thông, trường Đại học Công nghệ Thông tin – Đại học Quốc Gia Hồ Chí Minh.

Mặc dù trong quá trình làm đồ án, nhóm chúng em đã rất cố gắng, tuy nhiên cũng không tránh khỏi những thiếu sót. Chúng em hy vọng rằng sẽ nhận được những nhận xét, góp ý của thầy/cô về những vấn đề được triển khai trong bài báo cáo đồ án này, giúp chúng em có thể hoàn thiện và có những kinh nghiệm quý báu cho những đồ án tiếp theo.

Cuối cùng, chúng em xin kính chúc thầy/cô luôn dồi dào sức khỏe và thành công trong sự nghiệp giảng dạy cao quý.

Chúng em xin chân thành cảm ơn!

Thành phố Hồ Chí Minh, ngày 22 tháng 10 năm 2025

Nhóm 20

1. TỔNG QUAN

1.1. Bối cảnh thực tế

Trong những năm gần đây, xu hướng chuyển dịch hạ tầng công nghệ từ mô hình truyền thống sang mô hình điện toán đám mây đang diễn ra một cách mạnh mẽ trong nhiều doanh nghiệp ở nhiều quy mô. Việc ứng dụng công nghệ đám mây giúp mang lại cho hệ thống khả năng mở rộng linh hoạt, tối ưu chi phí đầu tư hạ tầng và nâng cao hiệu quả vận hành. Đặc biệt, Kubernetes đã trở thành nền tảng tiêu chuẩn để triển khai và quản lý các ứng dụng container hóa nhờ khả năng tự động hóa, cân bằng tải và đảm bảo tính sẵn sàng cao cho dịch vụ.

Tuy nhiên, cùng với sự phát triển và phổ biến rộng rãi của Kubernetes, các rủi ro bảo mật cũng ngày càng gia tăng, trong đó sai lệch cấu hình trong quá trình triển khai và vận hành là nguyên nhân phổ biến nhất. Kubernetes là một hệ thống vô cùng phức tạp với nhiều thành phần liên kết chặt chẽ, nếu chúng ta cấu hình không phù hợp dù chỉ một bước cũng có thể trở thành điểm yếu nghiêm trọng, tạo điều kiện cho các cuộc tấn công leo thang đặc quyền, rò rỉ dữ liệu hoặc gây gián đoạn dịch vụ. Để giảm thiểu gánh nặng quản trị và áp dụng các quy trình tốt, Google Kubernetes Engine (GKE), một dịch vụ quản lý Kubernetes của Google Cloud đã cung cấp chế độ Autopilot với cơ chế quản lý tự động, tuy nhiên điều này cũng đặt ra yêu cầu cần có một bộ tiêu chí chuẩn hóa nhằm đánh giá mức độ an toàn của hệ thống.

Để khắc phục điều đó, đề án của nhóm được thực hiện nhằm đánh giá mức độ tuân thủ bảo mật của cụm GKE theo chuẩn CIS GKE Autopilot Benchmark, tập trung vào các khuyến nghị thuộc Level 1, nhóm kiểm soát bảo mật cơ bản, dễ triển khai và ít ảnh hưởng đến hoạt động vận hành. Trong đề án, các công việc được thực hiện liên quan đến phân tích các cấu hình bảo mật quan trọng, xác định những điểm chưa đáp ứng yêu cầu của CIS Benchmark và đề xuất các biện pháp khắc phục phù hợp nhằm nâng cao mức độ an toàn cho cụm Kubernetes.

Phạm vi nghiên cứu của đề án được giới hạn trong việc đánh giá trên hai chế độ triển khai GKE là Standard và Autopilot, tập trung vào các nhóm kiểm soát chính như RBAC và Service Account, Pod Security Standards, quản lý namespace, và cấu hình mạng bao gồm VPC Flow Logs và Intra-node Visibility.

1.2. Giới thiệu

1.2.1. Tổng quan về Kubernetes

1.2.1.1 Định nghĩa

Kubernetes, hay thường được viết tắt là K8s, là một hệ thống mã nguồn mở đóng vai trò quan trọng trong việc tự động hóa quy trình triển khai, mở rộng và quản lý các ứng dụng dưới dạng container. Ngoài ra, Kubernetes giúp cho chúng ta đơn giản hóa cách vận hành các hệ thống phân tán, đồng thời nâng cao tính linh hoạt, khả năng mở rộng và độ tin cậy của ứng dụng trong môi trường điện toán đám mây.



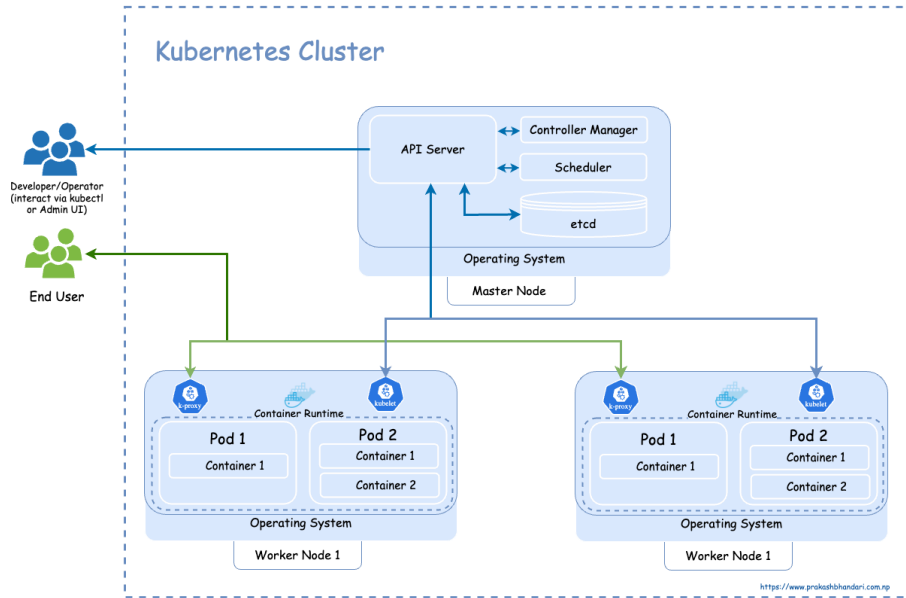
Hình 1. Logo Kubernetes

Các chức năng ưu việt của Kubernetes có thể kể đến bao gồm:

- **Mở rộng linh hoạt:** Cho phép tự động mở rộng hoặc thu hẹp ứng dụng dựa trên nhu cầu sử dụng mà không làm gián đoạn dịch vụ.
- **Cân bằng tải và phát hiện dịch vụ:** Tự động phân phối lưu lượng truy cập và hỗ trợ phát hiện các dịch vụ trong cụm.
- **Khả năng phục hồi:** Tự động thay thế hoặc khởi động lại các container gặp sự cố, đảm bảo ứng dụng luôn ở trạng thái sẵn sàng cao.
- **Triển khai và phục hồi tự động:** Hỗ trợ triển khai các bản cập nhật của ứng dụng và khôi phục phiên bản trước khi xảy ra lỗi.

1.2.1.2 Kiến trúc Kubernetes

Kiến trúc của Kubernetes được thiết kế theo mô hình client–server, bao gồm hai thành phần chính: Control plane (Master Node) và Worker Node, phối hợp cùng nhau để quản lý và vận hành các ứng dụng trong cụm.



Hình 2. Mô hình kiến trúc K8s

Control plane chịu trách nhiệm quản lý toàn bộ trạng thái và điều phối hoạt động của cụm Kubernetes. Thành phần này bao gồm các dịch vụ cốt lõi như:

- **API Server:** Tiếp nhận và xử lý các yêu cầu từ người dùng và các thành phần trong hệ thống.
- **Scheduler:** Đảm nhiệm việc phân bổ Pod lên các Worker node phù hợp.
- **Controller manager:** Theo dõi và đảm bảo trạng thái thực tế của hệ thống khớp với trạng thái mong muốn.
- **etcd:** Cơ sở dữ liệu phân tán dùng để lưu trữ toàn bộ cấu hình và trạng thái của cụm.

Worker Node là nơi trực tiếp chạy các ứng dụng container hóa. Mỗi Worker Node bao gồm:

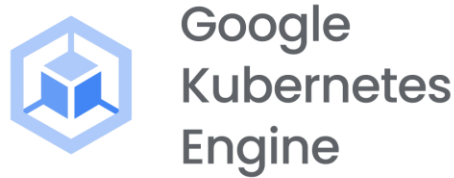
- **kubelet:** Để giao tiếp với Control Plane và quản lý vòng đời của Pod.
- **kube-proxy:** Xử lý lưu lượng mạng và cân bằng tải nội bộ, cùng với container runtime chịu trách nhiệm khởi chạy và quản lý container.

Sự phân tách rõ ràng giữa Control Plane và Worker Node giúp Kubernetes có thể dễ dàng mở rộng, tăng khả năng chịu lỗi và đảm bảo hệ thống hoạt động ổn định ngay cả khi một số thành phần gặp sự cố.

1.2.2. Tổng quan về Google Kubernetes Engine (GKE)

1.2.2.1 Định nghĩa GKE

Google Kubernetes Engine (GKE), là dịch vụ Kubernetes được quản lý bởi Google Cloud, cho phép chúng ta triển khai, quản lý và mở rộng các ứng dụng container hóa mà không cần trực tiếp quan tâm đến việc vận hành hạ tầng bên dưới. GKE kế thừa đầy đủ các đặc điểm của Kubernetes, đồng thời tích hợp chặt chẽ với hệ sinh thái Google Cloud nhằm đảm bảo tính ổn định, bảo mật và khả năng mở rộng.



Hình 3. Logo GKE

GKE hỗ trợ tự động hóa các tác vụ quan trọng như tạo và nâng cấp cluster, quản lý tài nguyên, cân bằng tải, cũng như tích hợp sẵn các cơ chế bảo mật, giám sát và ghi log. Nền tảng này cung cấp hai chế độ triển khai là Standard và Autopilot, cho phép người dùng lựa chọn giữa mức độ kiểm soát hạ tầng linh hoạt hoặc mô hình quản lý tự động theo các thực hành tốt nhất, phù hợp với nhiều nhu cầu vận hành khác nhau.

1.2.2.2 So sánh Standard và Autopilot

Ở chế độ GKE Standard, người dùng có toàn quyền quản lý cụm Kubernetes, bao gồm cấu hình node, mạng, bảo mật và các thành phần hỗ trợ. Mô hình này mang lại tính linh hoạt cao, phù hợp với các hệ thống yêu cầu tùy chỉnh sâu, tuy nhiên đòi hỏi người quản trị phải có kiến thức chuyên môn và chịu trách nhiệm lớn trong việc đảm bảo an toàn và vận hành ổn định.

Ngược lại, GKE Autopilot hướng đến việc đơn giản hóa quản trị bằng cách để Google Cloud tự động quản lý hạ tầng và áp dụng các thực hành tốt mặc định. Chúng ta chỉ cần tập trung vào việc triển khai workload, trong khi các cấu hình liên quan đến node, tài nguyên và nhiều chính sách bảo mật được hệ thống tự động thiết lập. Mô hình này giúp giảm rủi ro sai lệch cấu hình và nâng cao mức độ an toàn, tuy nhiên mức độ tùy chỉnh bị giới hạn so với chế độ Standard.

1.2.3 Giới thiệu về CIS và CIS Benchmark

Center for Internet Security, hay có tên viết tắt là CIS, là một tổ chức phi lợi nhuận với nhiệm vụ xây dựng các tiêu chuẩn bảo mật toàn cầu cho hạ tầng công nghệ thông tin. CIS tập trung vào việc cung cấp các hướng dẫn, công cụ và tiêu chuẩn giúp các tổ chức giảm thiểu rủi ro trong an ninh mạng, trong đó CIS Benchmarks™ là một trong những bộ tiêu chuẩn nổi bật và được sử dụng rộng rãi nhất trong các hệ thống, nền tảng và dịch vụ đám mây.



Hình 4. Logo của CIS

CIS Benchmarks là tài liệu tập trung vào các cài đặt cấu hình kỹ thuật được sử dụng để duy trì hoặc tăng cường bảo mật cho công nghệ được đề cập. Các Benchmark này được thiết kế để trở thành một thành phần cốt lõi của một chương trình an ninh mạng toàn diện.

Quá trình phát triển các CIS Benchmark được thực hiện thông qua quy trình đánh giá đồng thuận với sự tham gia của một cộng đồng các chuyên gia chủ đề toàn cầu. Quá trình này kết hợp kinh nghiệm thực tế với thông tin dựa trên dữ liệu để tạo ra hướng dẫn mang tính quy tắc cụ thể theo công nghệ nhằm hỗ trợ người dùng bảo mật môi trường của họ.

1.2.4 Giới thiệu CIS Google Kubernetes Engine Autopilot Benchmark

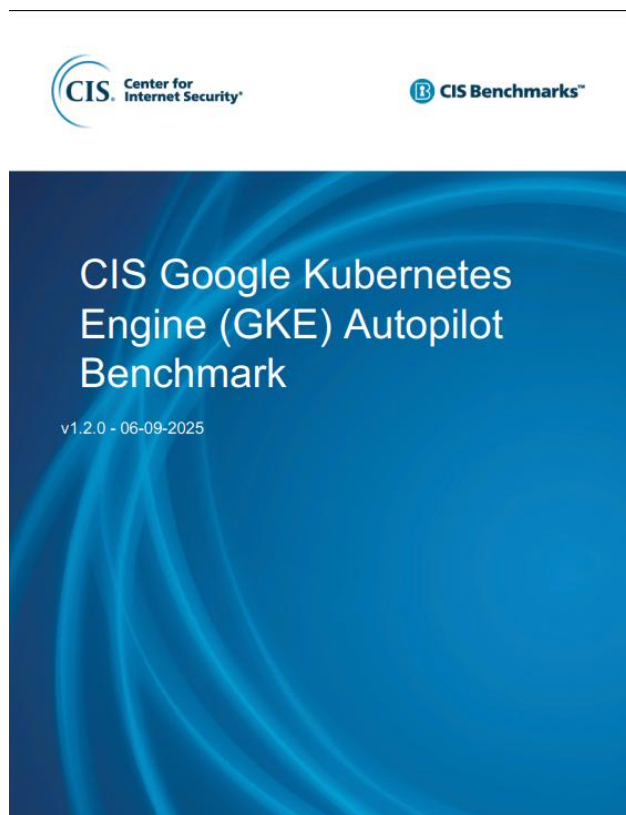
Cuốn CIS Google Kubernetes Engine (GKE) Autopilot Benchmark v1.2.0, được phát hành vào tháng 6 năm 2025, là tài liệu hướng dẫn cấu hình bảo mật chuyên biệt cho nền tảng Google Kubernetes Engine (GKE) ở chế độ Autopilot. Tài liệu này tập trung vào những thiết lập mà người dùng cuối có thể kiểm soát để đảm bảo tính bảo mật, tính tuân thủ, và khả năng kiểm toán khi vận hành cụm Kubernetes trên Google Cloud.

Trong Benchmark, các khuyến nghị được chia thành hai cấp độ:

- **Level 1:** Bao gồm các thiết lập bảo mật cơ bản, mang tính thiết thực và dễ áp dụng, nhằm đảm

bảo an toàn mà không ảnh hưởng đáng kể đến hiệu năng và tính ổn định của hệ thống. Đây là cấp độ phù hợp cho phần lớn môi trường doanh nghiệp hoặc hệ thống sản xuất tiêu chuẩn.

- **Level 2:** Bao gồm các thiết lập nâng cao và chuyên sâu hơn, hướng đến việc tăng cường mức độ bảo mật cho các hệ thống đòi hỏi độ tin cậy cao hoặc có dữ liệu nhạy cảm. Một số khuyến nghị ở Level 2 có thể yêu cầu thay đổi sâu về cấu hình hoặc ảnh hưởng đến khả năng vận hành, nên cần được xem xét kỹ trước khi áp dụng.



Hình 5. CIS Google Kubernetes Engine (GKE) Autopilot Benchmark v1.2.0

Nhờ sự phân cấp rõ ràng này, người quản trị có thể lựa chọn áp dụng Benchmark phù hợp với mức độ rủi ro, quy mô và yêu cầu bảo mật của tổ chức, từ đó xây dựng nên một hệ thống GKE an toàn, nhất quán và tuân thủ các chuẩn về an ninh của quốc tế.

1.3. Nội dung Benchmark

CIS GKE Autopilot Benchmark được cấu trúc xoay quanh hai nhóm nội dung trọng tâm: Policies và Managed Services. Hai phần này bao quát toàn bộ các khía cạnh bảo mật quan trọng của một cụm Kubernetes trong môi trường Autopilot, từ việc kiểm soát truy cập nội bộ đến việc quản lý an toàn các dịch vụ đám mây tích hợp. Nhờ cách phân tách minh bạch này, người quản trị có thể dễ dàng triển khai, đánh giá và duy trì cấu hình bảo mật cho từng lớp của hệ thống.

1.3.1 Policies

Phần này tập trung vào các chính sách bảo mật trong cụm GKE Autopilot, bao gồm kiểm soát truy cập dựa trên vai trò người dùng (RBAC), quản lý tài khoản dịch vụ, chuẩn bảo mật cho Pod, cấu hình mạng, và quản lý Secrets. Các chính sách được thiết kế nhằm đảm bảo quyền truy cập tối thiểu, cô lập tài nguyên, và ngăn chặn các hành vi cố gắng xâm nhập trái phép trong môi trường Kubernetes. Ngoài ra, phần này cũng đề cập đến các cơ chế như Namespace để tăng cường tính an toàn cho toàn bộ cụm.

1.3.1.1 Phân quyền dựa trên vai trò (RBAC) và Tài khoản dịch vụ (Service Account)

Ensure that the cluster-admin role is only used where required (Automated):

- Dịch: Đảm bảo vai trò cluster-admin chỉ được sử dụng khi cần thiết (Tự động hóa)
- Ý nghĩa: Vai trò *cluster-admin* cung cấp đặc quyền quản trị cao nhất trong toàn bộ cụm Kubernetes. Mục đích của việc hạn chế sử dụng vai trò này giúp giảm thiểu nguy cơ lạm dụng quyền và ngăn chặn các tác động không mong muốn đến hệ thống. Các tài khoản chỉ nên được gán *cluster-admin* khi thực sự cần thiết cho các hoạt động quản trị chủ yếu.

Minimize wildcard use in Roles and ClusterRoles (Automated):

- Dịch: Giảm thiểu việc sử dụng ký tự đại diện (wildcard) trong Roles và ClusterRoles.
- Ý nghĩa: Kubernetes Roles (vai trò) và ClusterRoles (vai trò cấp cụm) cung cấp quyền truy cập vào các tài nguyên dựa trên các tập hợp đối tượng và hành động có thể thực hiện được gán cho các đối tượng đó. Có thể đặt một trong hai thành ký tự đại diện “*” (wildcard), khớp với tất cả các mục. Khi loại bỏ hoặc hạn chế wildcard giúp đảm bảo rằng từng quyền được ủy quyền một cách cụ thể và có thể kiểm soát được. Điều này cũng hỗ trợ công việc kiểm toán và giám sát an ninh một cách dễ dàng hơn.

Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster (Manual):

- Dịch: Giới hạn việc sử dụng quyền Bind, Impersonate và Escalate trong Kubernetes cluster.
- Ý nghĩa: ClusterRole và các Role có quyền Impersonate (mạo danh), Bind (liên kết) hoặc Escalate (leo thang) không nên được cấp trừ khi có yêu cầu nghiêm ngặt. Chỉ nên gán các quyền này khi có yêu cầu vận hành đặc biệt và được giám sát một cách chặt chẽ. Kiểm soát cẩn thận nhóm quyền này là cần thiết để bảo vệ hệ thống khỏi các hành vi leo thang đặc quyền.

1.2.1.4 Tiêu chuẩn bảo mật Pod

Ensure that the cluster enforces Pod Security Standard Baseline profile or stricter for all namespaces. (Manual):

- Dịch: Đảm bảo cụm áp dụng và thực thi hồ sơ Pod Security Standard Baseline hoặc nghiêm ngặt hơn cho tất cả các namespace.
- Ý nghĩa: Pod Security Standard Baseline cung cấp bộ quy tắc bảo mật tối thiểu để ngăn chặn cấu hình Pod không an toàn. Áp dụng quy tắc Baseline (hoặc cao hơn) cho toàn bộ namespace giúp giảm thiểu các tình huống rủi ro như chạy container đặc quyền hoặc mount volume mang tính chất nhạy cảm. Đây là bước bắt buộc để duy trì mức an toàn nhất quán cho toàn cụm.

1.2.1.3 Các chính sách tổng quát

Create administrative boundaries between resources using namespaces (Manual):

- Dịch: Tạo ranh giới quản trị giữa các tài nguyên bằng cách sử dụng namespaces.
- Ý nghĩa: Namespaces cho phép phân tách tài nguyên một cách logic trong cụm, giúp kiểm soát quyền truy cập và giới hạn phạm vi quản trị. Việc sử dụng namespaces làm tăng tính cô lập giữa ứng dụng và hỗ trợ triển khai mô hình RBAC chi tiết hơn. Đây là một trong những phương pháp quan trọng, an toàn để đảm bảo cấu trúc quản lý nhất quán và an toàn.

1.3.2 Managed Services

Chương này tập trung vào các dịch vụ được quản lý trong GKE Autopilot, bao gồm Image Registry, IAM, Cloud KMS, Network, Authentication, Storage, và các cấu hình cụm khác. Mục tiêu là hướng dẫn người dùng các phương pháp đảm bảo an toàn cho các thành phần tích hợp với Google Cloud, chẳng hạn như bật quét lỗ hổng Image, mã hóa Secrets bằng Cloud KMS, hay kiểm soát quyền truy cập IAM.

1.3.2.1 Mạng lưới trong cụm (Cluster Networking)

Enable VPC Flow Logs and Intranode Visibility:

- Dịch: Bật VPC Flow Logs và khả năng quan sát lưu lượng nội bộ giữa các node.
- Ý nghĩa: VPC Flow Logs cung cấp khả năng giám sát lưu lượng mạng để phục vụ việc phân tích, điều tra và đảm bảo tính an ninh. Intranode Visibility cho phép quan sát lưu lượng nội bộ giữa các Pod trong cùng node, đảm bảo không có lưu lượng bị bỏ sót. Kích hoạt cả hai tính năng giúp tăng cường khả năng giám sát và đáp ứng sự cố mạng.

2. PHƯƠNG PHÁP

2.1 Manual

Một số khuyến nghị thuộc Level 1 không thể tự động hóa một cách hoàn toàn do bị phụ thuộc vào ngữ cảnh vận hành thực tế và mục đích sử dụng của hệ thống. Đối với các yêu cầu này, việc đánh giá cần dựa trên việc phân tích thủ công và xác minh trực tiếp từ phía quản trị viên để đảm bảo cấu hình đang được áp dụng đúng mục đích và không vượt quá nhu cầu cần thiết.

Sử dụng Google Cloud SDK Shell để:

- Rà soát quyền truy cập đặc biệt như Bind, Impersonate, Escalate.
- Kiểm tra namespace, phân quyền RBAC, cấu trúc tài nguyên và việc sử dụng workload.
- Xác minh thủ công bằng các lệnh tương tác với cụm Kubernetes như `kubectl get`, `kubectl describe`, và kiểm tra log để bảo đảm không có cấu hình vượt mức hoặc sử dụng sai mục đích.
- So sánh thực trạng hệ thống với yêu cầu Level 1 để xác định tuân thủ.

2.2 Automation

Phương pháp Automation được áp dụng đối với các khuyến nghị có thể can thiệp và kiểm soát trực tiếp thông qua cấu hình hệ thống, công cụ DevOps hoặc các dịch vụ tích hợp sẵn của Google Kubernetes Engine. Mục tiêu của phương pháp này là đảm bảo tính nhất quán, khả năng tái tạo cấu hình và giảm thiểu rủi ro phát sinh từ thao tác thủ công trong quá trình triển khai và vận hành.

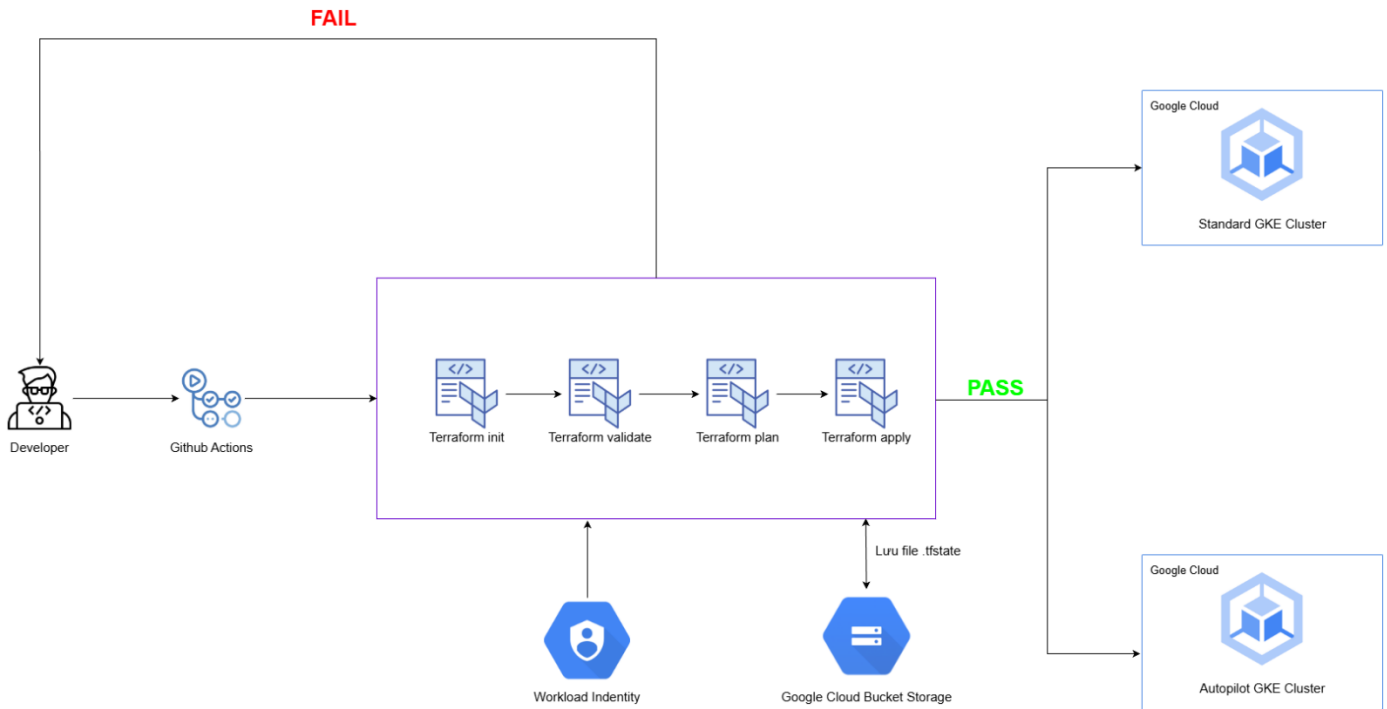
Các phương pháp Automation :

- Thiết lập cấu hình bảo mật bằng `kubectl`, `gcloud`, YAML manifest hoặc Terraform.
- Sử dụng các nhãn (label), chính sách mặc định và thông số cấu hình để áp dụng Pod Security, RBAC, và Service Account theo yêu cầu Level 1.
- Bật các tính năng giám sát (ví dụ: VPC Flow Logs, Intranode Visibility) trực tiếp bằng `gcloud` hoặc UI.

3. TRIỂN KHAI

3.1. Mô hình

3.1.1. Mô hình kiến trúc



Hình 6. Mô hình kiến trúc

Sơ đồ phía trên mô tả quy trình tự động hóa việc cung cấp hạ tầng trên Google Cloud Platform bằng Terraform (một công cụ cung cấp hạ tầng dưới dạng mã), được điều phối thông qua GitHub Actions. Quy trình bắt đầu từ nhà phát triển và kết thúc bằng việc triển khai hạ tầng Kubernetes trên GCP. Cách thực hiện này giúp cho hạ tầng đảm bảo tính nhất quán, bảo mật, khả năng kiểm soát phiên bản và tự động hóa toàn bộ vòng đời.

Chi tiết quy trình được miêu tả như sau:

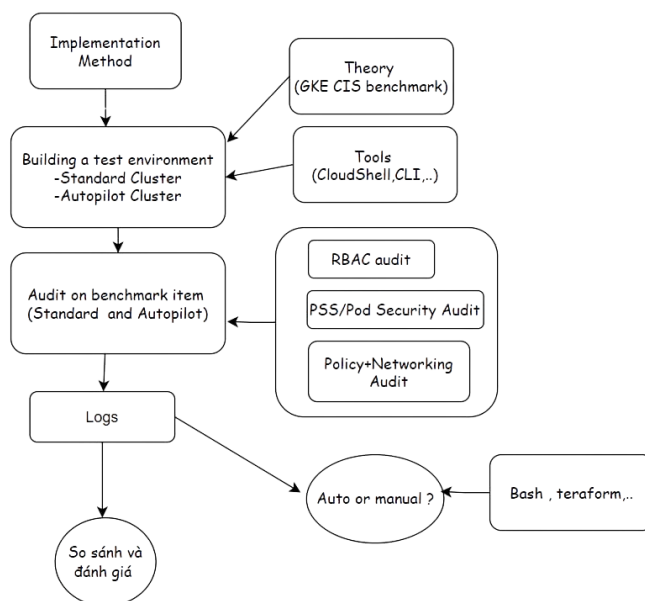
- **Bước 1: Kích hoạt quy trình:** Quy trình được kích hoạt khi Developer (nhà phát triển) thực hiện đẩy mã nguồn Terraform lên kho lưu trữ GitHub. Sự kiện này sẽ tự động khởi chạy pipeline CI/CD trên GitHub Actions.
- **Bước 2: Xác thực và quản lý trạng thái:**
 - *Xác thực:* GitHub Actions sử dụng Workload Identity để xác thực và truy cập tài nguyên Google Cloud. Cơ chế này loại bỏ việc sử dụng Service Account Key tĩnh, giúp tăng cường bảo mật và tuân theo các tiêu chuẩn an ninh của GCP.

- *Quản lý trạng thái Terraform*: File trạng thái Terraform (.tfstate) được lưu trữ tập trung trên Google Cloud Storage Bucket. Việc này giúp đảm bảo trạng thái hạ tầng luôn được đồng bộ giữa các lần chạy pipeline và hỗ trợ làm việc nhóm hiệu quả.
- **Bước 3: Thực thi pipeline Terraform**: Pipeline trong GitHub Actions lần lượt thực hiện bốn bước chính của Terraform:
 - *terraform init*: Khởi tạo môi trường làm việc, cấu hình backend và tải các provider cần thiết.
 - *terraform validate*: Kiểm tra cú pháp và tính hợp lệ của cấu hình Terraform.
 - *terraform plan*: Lập kế hoạch triển khai, cho phép xem trước các thay đổi sẽ được áp dụng lên hạ tầng.
 - *terraform apply*: Áp dụng kế hoạch và triển khai tài nguyên thực tế trên Google Cloud.
- **Bước 4: Xử lý luồng kết quả**
 - *FAIL*: Nếu bất kỳ bước nào trong pipeline thất bại (lỗi cấu hình, xác thực hoặc xung đột tài nguyên), pipeline sẽ dừng lại và trả lỗi về cho nhà phát triển để kiểm tra và khắc phục.
 - *PASS*: Nếu tất cả các bước, đặc biệt là terraform apply, thực thi thành công, hạ tầng sẽ được triển khai hoặc cập nhật trên Google Cloud.

Sau khi pipeline hoàn tất thành công, hệ thống sẽ tự động khởi tạo các cụm Kubernetes theo cấu hình đã định nghĩa, bao gồm:

- **Standard GKE Cluster**: Cụm Kubernetes tiêu chuẩn, trong đó người dùng trực tiếp quản lý các node.
- **Autopilot GKE Cluster**: Cụm Kubernetes được Google quản lý hoàn toàn về node, giúp giảm gánh nặng vận hành và tối ưu tài nguyên.

3.1.2 Sơ đồ luồng làm việc



Hình 7. Sơ đồ luồng làm việc

Quy trình phía trên mô tả phương pháp triển khai và đánh giá mức độ tuân thủ bảo mật của các cụm Google Kubernetes Engine (GKE) dựa trên tiêu chuẩn CIS Benchmark. Cách tiếp cận được xây dựng theo hướng kết hợp giữa cơ sở lý thuyết và các công cụ kiểm tra, nhằm so sánh và đánh giá hai mô hình GKE là Standard Cluster và Autopilot Cluster.

Quy trình đánh giá bao gồm các nội dung chính sau:

- **Phương pháp triển khai và cơ sở lý thuyết:** Việc đánh giá được thực hiện dựa trên bộ tiêu chuẩn GKE CIS Benchmark, làm nền tảng lý thuyết để xác định các tiêu chí bảo mật cần kiểm tra và đối chiếu trong quá trình kiểm chứng.
- **Công cụ sử dụng:** Các công cụ như Google Cloud Shell, gcloud CLI, kubectl và các tiện ích hỗ trợ khác được sử dụng để thu thập cấu hình, kiểm tra trạng thái bảo mật và trích xuất dữ liệu từ các cụm GKE.
- **Xây dựng môi trường thử nghiệm:** Hai môi trường GKE được triển khai song song nhằm phục vụ mục đích so sánh:
 - *Standard GKE Cluster:* Cho phép người dùng chủ động quản lý node và cấu hình bảo mật.
 - *Autopilot GKE Cluster:* Được Google quản lý tự động node, tập trung vào tối ưu vận hành và bảo mật mặc định.

- **Thực hiện audit theo CIS Benchmark:** Trên mỗi cụm, quá trình audit được tiến hành theo các nhóm tiêu chí chính:
 - *RBAC Audit:* Đánh giá cơ chế phân quyền, vai trò và nguyên tắc đặc quyền tối thiểu.
 - *Pod Security / PSS Audit:* Kiểm tra chính sách bảo mật pod và các ràng buộc an toàn khi chạy container.
 - *Policy và Networking Audit:* Đánh giá Network Policy và khả năng kiểm soát luồng truy cập giữa các thành phần trong cluster.
- **Thu thập log và dữ liệu đánh giá:** Kết quả kiểm tra và log hệ thống được ghi nhận nhằm phản ánh mức độ tuân thủ của từng tiêu chí bảo mật trên mỗi loại cụm.
- **Hình thức thực hiện audit:** Quá trình audit có thể được triển khai:
 - *Tự động:* Thông qua Bash script, Terraform hoặc các công cụ tự động hóa.
 - *Thủ công:* Kiểm tra trực tiếp cấu hình và kết quả thực thi thông qua CLI khi cần phân tích chi tiết.
- **So sánh và đánh giá kết quả:** Dữ liệu thu thập được sử dụng để so sánh mức độ tuân thủ CIS Benchmark giữa Standard GKE và Autopilot GKE, từ đó đánh giá ưu điểm, hạn chế và mức độ phù hợp của từng mô hình.

3.2. Khởi tạo cụm

3.2.1 Tạo cụm GKE Autopilot

3.2.1.1 Phương pháp Manual (tạo bằng UI)

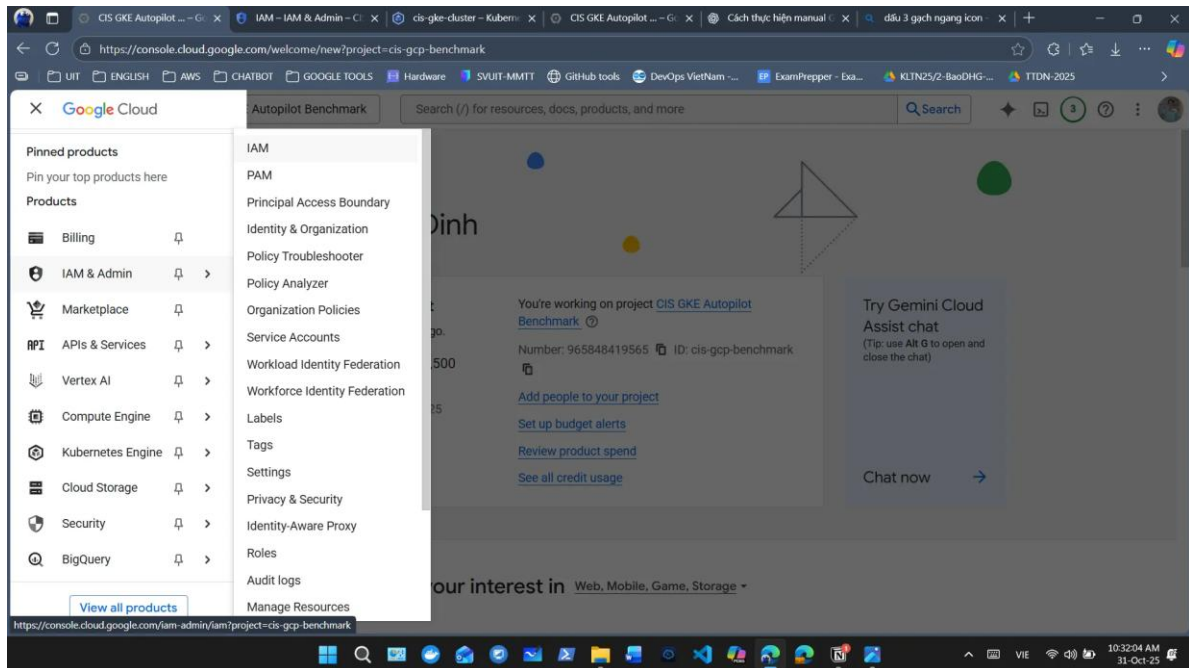
Tạo project

- Đăng nhập tài khoản GCP ở đường dẫn sau: [Google Cloud Platform](#)
- Tạo project mới có tên *CIS GKE Autopilot Benchmark*.

Quản lý quyền người dùng:

- Khi tạo một project mới trong Google Cloud, tài khoản tạo project sẽ mặc định có vai trò Owner, tức là sở hữu toàn quyền quản trị (bao gồm quản lý tài nguyên, phân quyền, và cấu hình bảo mật).

- Để thêm thành viên mới vào project, thực hiện như sau:
 - Nhấn vào biểu tượng  (Navigation menu) ở góc trên bên trái màn hình.
 - Chọn **IAM & Admin** → **IAM** để mở trang quản lý thành viên và phân quyền truy cập cho project.



Hình 8. IAM & Admin

- Chọn Grant Access để tiến hành cấp quyền truy cập cho thành viên mới.
- Nhập các thông tin cần thiết như sau:

Principal ?

xuancuongchyz1@gmail.com

Project

CIS GKE Autopilot Benchmark

Assign roles

Roles are composed of sets of permissions and determine what the principal can do with this resource. [Learn more](#)

Role

Dev Ops

Enables DevOps users to build and deploy applications, create, manage and perform administrative tasks on associated GCP resources

IAM condition (optional) ?

+ Add IAM condition

Role

Kubernetes Engine Develop...

Full access to Kubernetes API objects inside Kubernetes Clusters.

IAM condition (optional) ?

+ Add IAM condition

+ Add another role

Help me choose roles

Save

Test changes

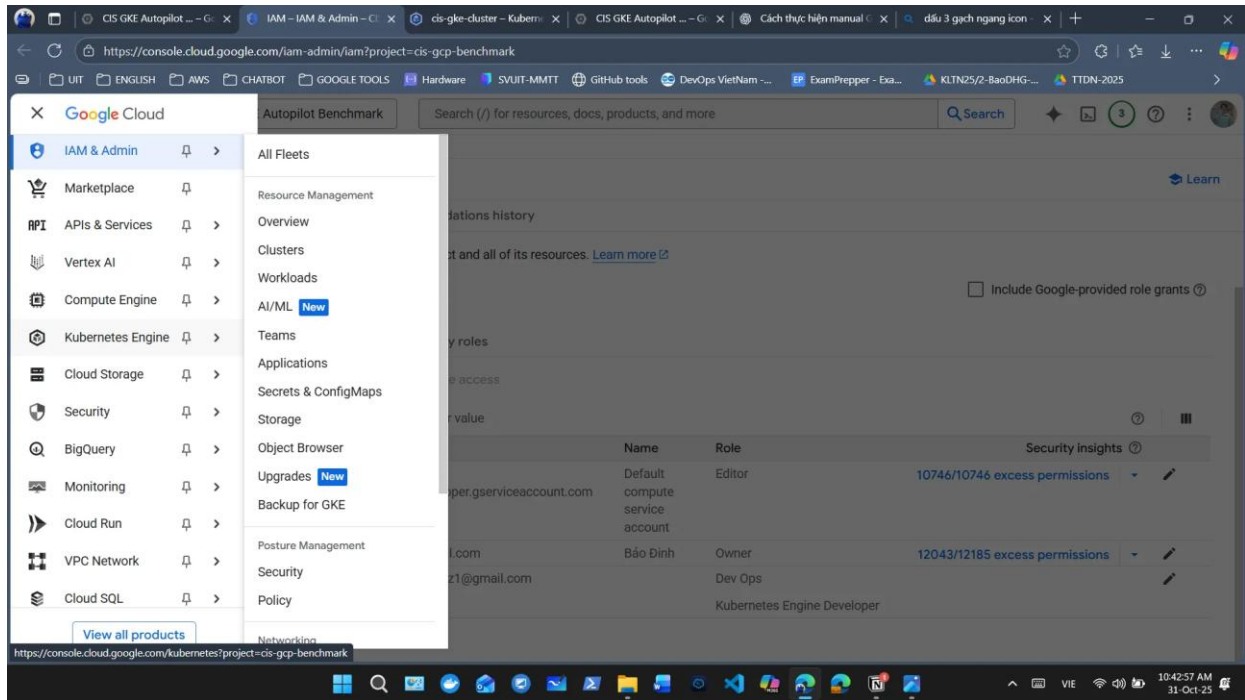
Cancel

Hình 9. Nhập role cần thiết

- Principal:
 - Là đối tượng được cấp quyền truy cập (user, service account, hoặc group).
 - Ở đây: xuancuongchyz1@gmail.com chính là user cá nhân đang được phân quyền.
- Project:
 - Là phạm vi áp dụng quyền (scope).
 - Mọi quyền gán ở đây áp dụng cho toàn bộ project CIS GKE Autopilot Benchmark.
 - Bao gồm các dịch vụ trong project: GKE, IAM, Compute, Storage, v.v.
- Assign Roles:
 - Là danh sách các vai trò (IAM Roles) mà principal được gán.
 - Mỗi role chứa tập quyền (permissions) xác định hành động được phép thực hiện.
 - Một principal có thể có nhiều role cùng lúc, chính vì thế quyền sẽ được **cộng dồn**.
 - ◆ Role 1: DevOps
 - Cho phép build, deploy và quản lý ứng dụng trên GCP.
 - Bao gồm quyền thao tác với: Cloud Build, Cloud Deploy, Artifact Registry, Compute Engine, Cloud Run, Cloud Storage (liên quan deploy), thích hợp cho DevOps Engineer hoặc CI/CD pipeline.
 - ◆ Role 2: Kubernetes Engine Developer
 - Cung cấp đầy đủ quyền truy cập với Kubernetes API objects trong cluster (pods, deployments, services, ...).
 - Cho phép thao tác tài nguyên trong GKE nhưng không tạo/xóa cluster.
 - Thích hợp cho developer hoặc operator trong môi trường GKE.
- IAM Condition (Optional)
 - Cho phép thêm điều kiện ràng buộc khi cấp quyền:
 - Theo thời gian (hiệu lực đến ngày X).
 - Theo địa chỉ IP (chỉ cho phép từ IP nội bộ).
 - Theo resource attribute (ví dụ: chỉ cho resource có label env=prod).
- Nếu không có điều kiện, quyền có hiệu lực toàn bộ, không giới hạn.

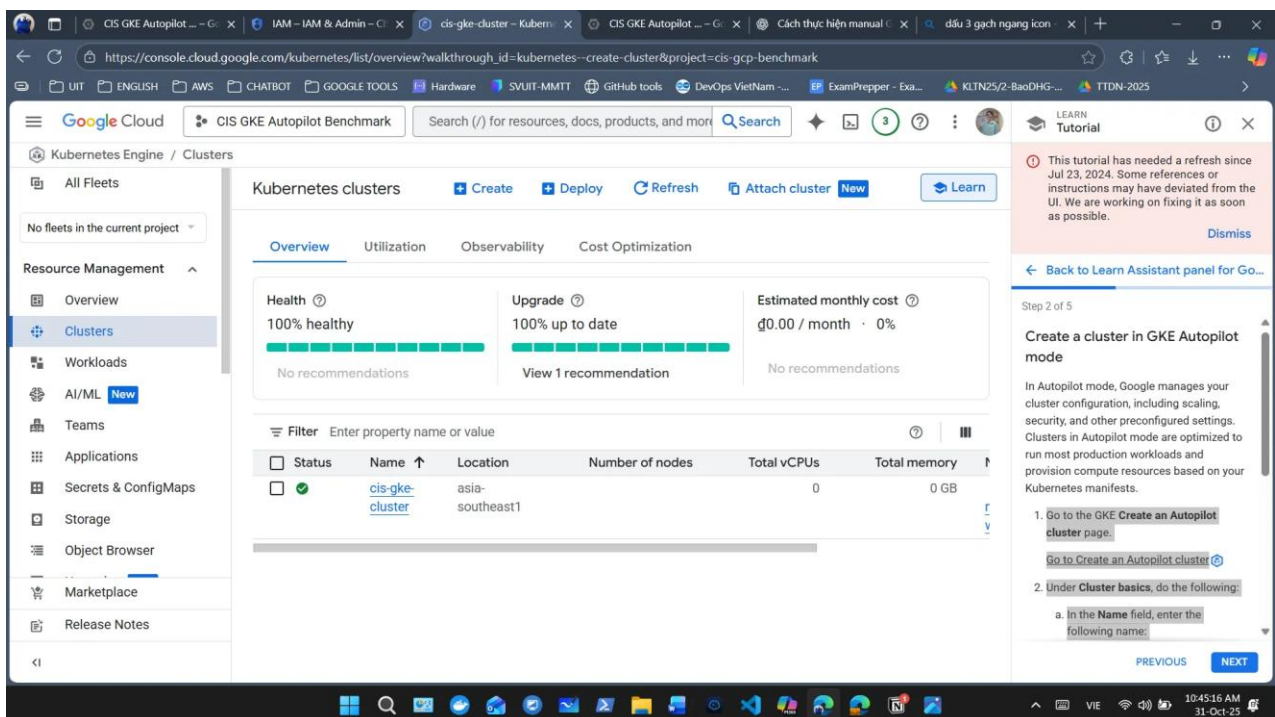
Tạo cụm GKE

- Mở menu ☰ ở góc trái giao diện Google Cloud Console, chọn Kubernetes Engine.



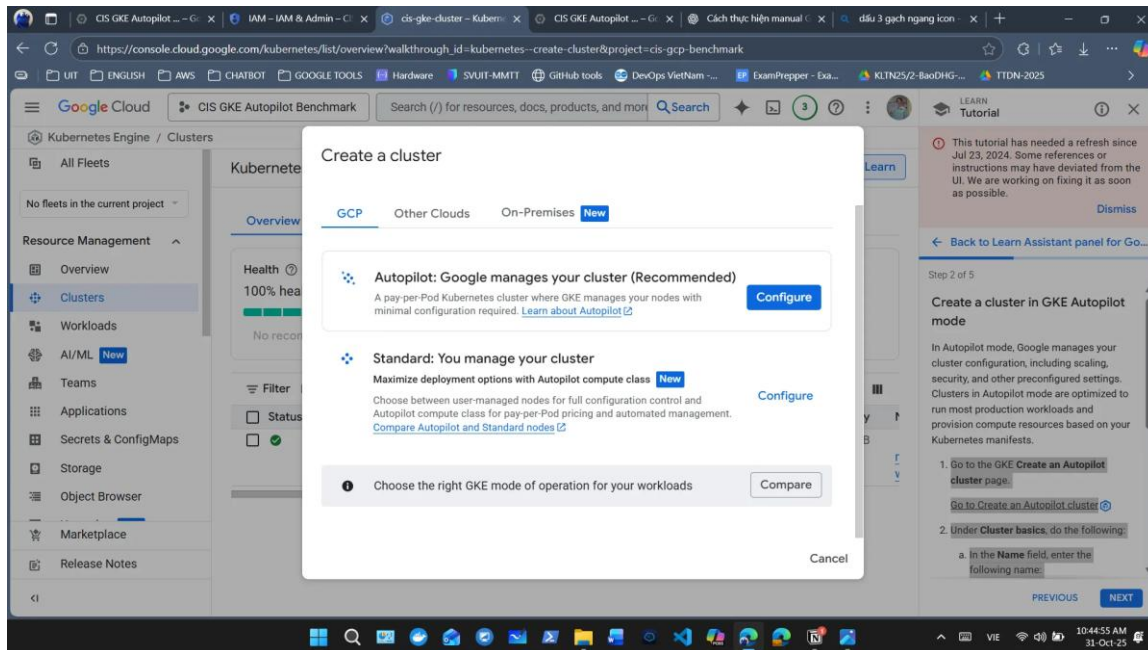
Hình 10. Chọn Kubernetes Engine

- Chọn dự án Google Cloud sẽ triển khai cụm GKE (nếu có sẵn): CIS GKE Autopilot Benchmark. Đảm bảo Kubernetes API đã được kích hoạt trước khi tiếp tục.



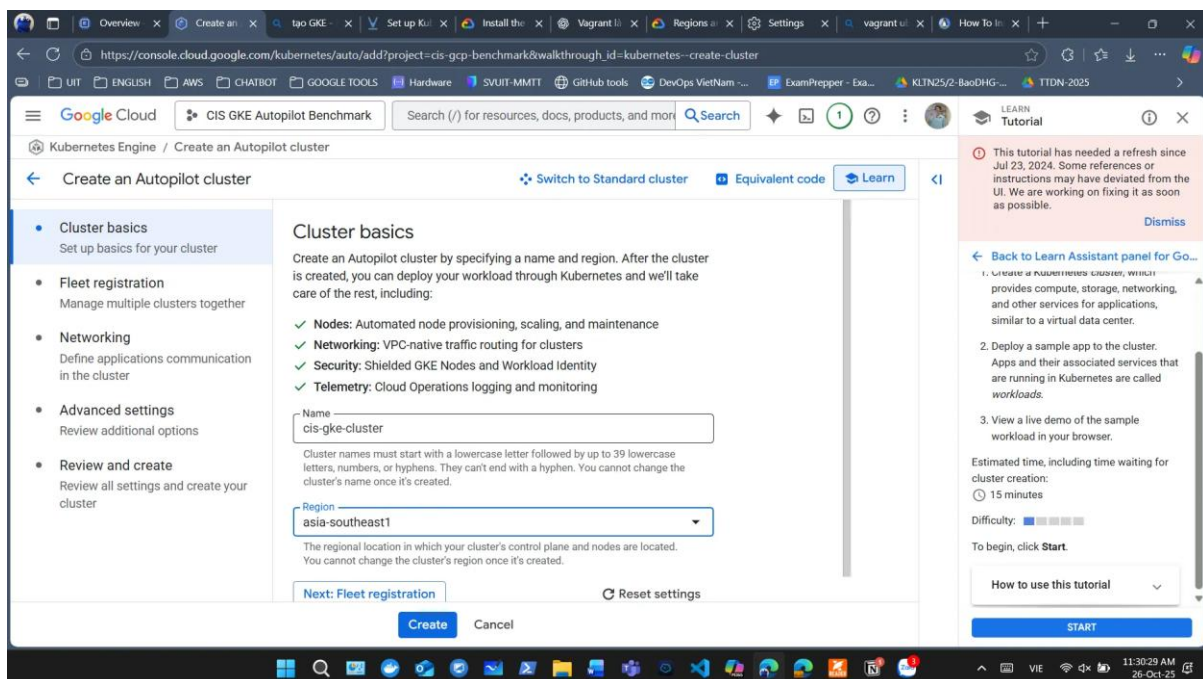
Hình 11. Giao diện tổng quát cụm Kubernetes

- Trong giao diện GKE, chọn mục **Overview** trong thanh menu để bắt đầu tạo cụm GKE. Nhấn **Create** → **Autopilot: Google manages your cluster (Recommended)** để tạo cụm → Chọn **Configure**.



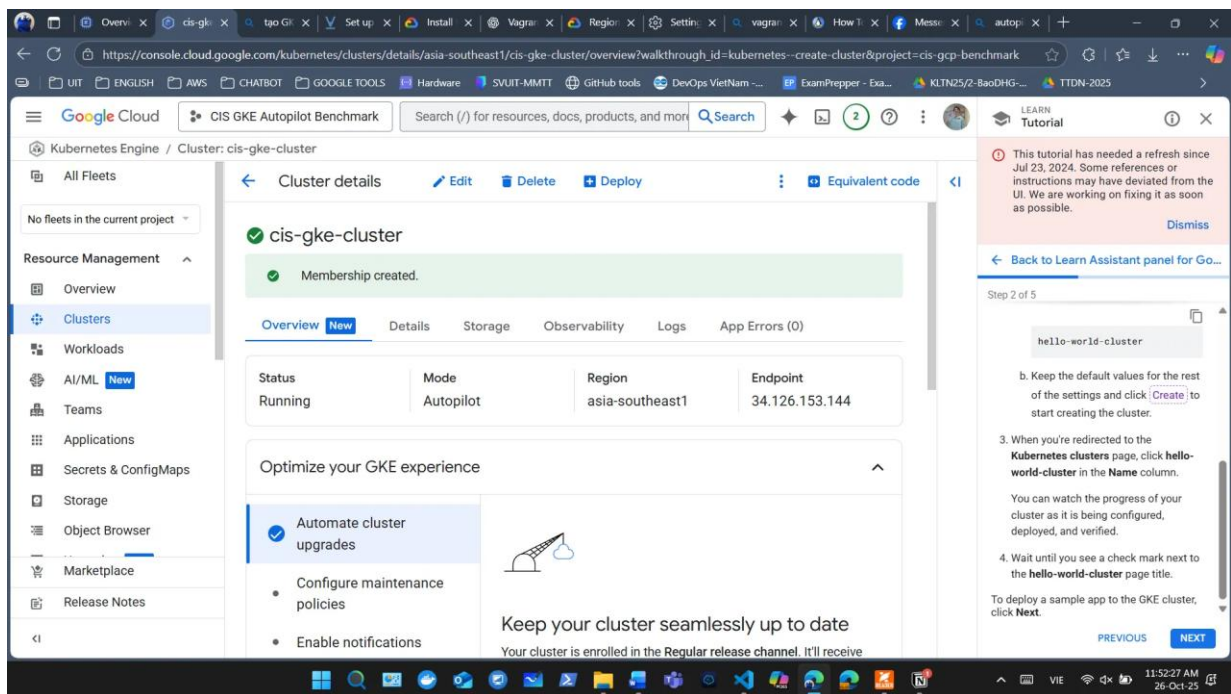
Hình 12. Chọn Autopilot

- Trong trường **Name**, nhập tên cụm: cis-gke-cluster. Trong trường **Region**, chọn vị trí triển khai: asia-southeast1 (Singapore).



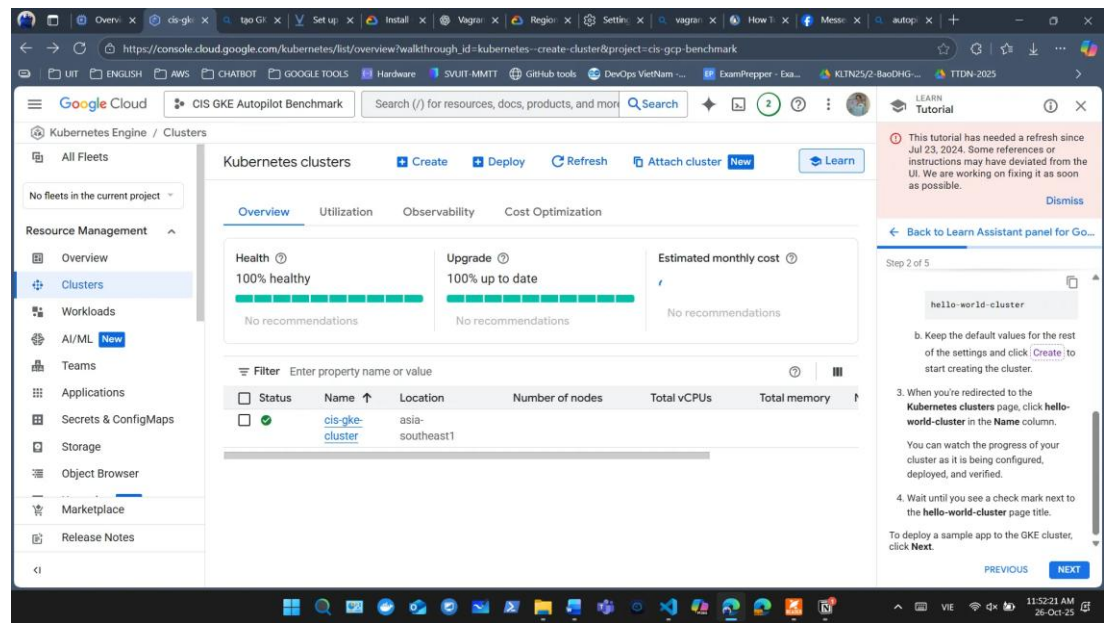
Hình 13. Nhập thông tin cụm Kubernetes Autopilot

- Giữ nguyên các giá trị mặc định khác trong phần Cluster basics. Nhấn Create để bắt đầu quá trình tạo cụm GKE Autopilot.
- Thông tin bổ sung hiển thị trong giao diện:
 - **Nodes:** Tự động cung cấp, mở rộng và bảo trì node.
 - **Networking:** Hỗ trợ định tuyến VPC-native.
 - **Security:** Bật Shielded GKE Nodes và Workload Identity.
 - **Telemetry:** Tích hợp Cloud Operations để thu thập log và giám sát.
- Sau khi hoàn tất, giao diện sẽ hiển thị tiến trình **Autopilot quản lý cụm** (Configure → Deploying → Health checks), và kết quả cuối cùng là trang **Overview** của cụm GKE đã tạo.



Hình 14. Hoàn tất tạo cụm Kubernetes Autopilot

- Đây là thông tin tổng quát của cụm:



Hình 15. Thông tin tổng quát cụm Autopilot

3.2.1.2 Phương pháp Automation

Tạo Github Actions workflow:

- Để đảm bảo quá trình triển khai hạ tầng Kubernetes của nhóm được diễn ra một cách tự động, nhất quán và có thể lặp lại, chúng em đã sử dụng GitHub Actions làm công cụ CI/CD cho việc triển khai hạ tầng dưới dạng mã. Luồng làm việc được thiết kế nhằm tự động thực thi các bước kiểm tra, khởi tạo và triển khai Terraform mỗi khi có thay đổi mã nguồn hoặc khi được kích hoạt thủ công.
- Thông qua GitHub Actions, toàn bộ quy trình từ xác thực với Google Cloud, khởi tạo Terraform backend, cho đến triển khai cụm GKE Autopilot đều được tự động hóa, giúp giảm thiểu thao tác thủ công, hạn chế lỗi cấu hình và nâng cao độ tin cậy trong quá trình triển khai hệ thống.
- Link cấu hình CI/CD cho cụm Kubernetes Autopilot: [Link cấu hình](#)

Tạo mã nguồn Terraform để triển khai hạ tầng dưới dạng mã:

```

gcp-gke-infr/autopilot/
├── backend.tf          # Remote state configuration (GCS)
├── main.tf             # GKE Autopilot cluster resource
├── provider.tf         # Terraform and Google provider config
├── terraform.tfvars    # Variable values (customize this)
├── variable.tf         # Input variable definitions
└── output.tf           # Output values after deployment

```

Hình 16. Cấu trúc thư mục Kubernetes Autopilot

- **Khởi tạo backend lưu trữ state từ xa:** Sử dụng tệp *backend.tf* để cấu hình remote state trên Google Cloud Storage (GCS). Mục đích của việc này để đảm bảo trạng thái của mã nguồn Terraform được lưu trữ an toàn, hỗ trợ làm việc nhóm và tránh xung đột trạng thái.
- **Cấu hình Terraform provider:** Thiết lập phiên bản của công cụ cần dùng trong file *provider.tf*.
- **Định nghĩa cụm GKE Autopilot:** Tài nguyên GKE Autopilot được khai báo trong *main.tf*.
- **Quản lý biến đầu vào:** Các biến cấu hình được định nghĩa tập trung trong *variable.tf*. Giá trị cụ thể của biến được gán thông qua *terraform.tfvars*, giúp dễ dàng thay đổi môi trường triển khai mà không ảnh hưởng đến mã nguồn chính.
- **Xuất thông tin sau triển khai:** Tệp *output.tf* được sử dụng để xuất các thông tin quan trọng của cụm sau khi tạo thành công.
- **Tự động hóa thông qua GitHub Actions:** Workflow GitHub Actions thực thi lần lượt các lệnh *terraform init*, *terraform validate*, *terraform plan*, và *terraform apply*. Toàn bộ quy trình được kích hoạt tự động, đảm bảo triển khai hạ tầng một cách nhất quán và có kiểm soát.

3.2.2 Tạo cụm GKE Standard

3.2.2.1 Phương pháp Manual (tạo bằng UI)

Bước 1: Chuẩn bị và truy cập Google Cloud Console

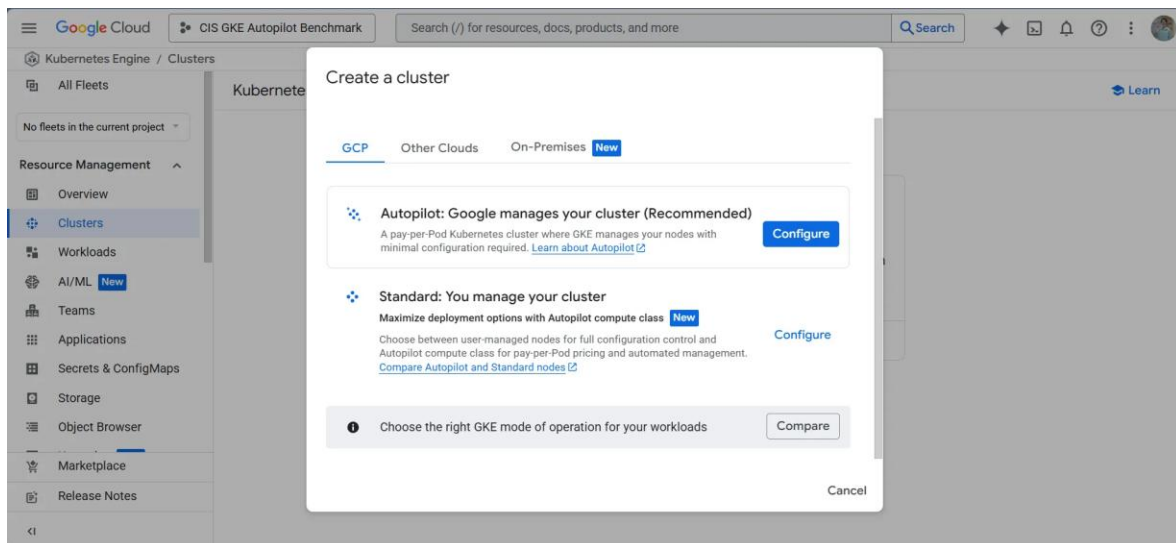
- Truy cập Google Cloud Console: Mở trình duyệt web và truy cập địa chỉ: console.cloud.google.com. Chọn dự án mà nhóm muốn tạo cụm GKE, ở đây sẽ là *CIS GKE Autopilot Benchmark*.

Bước 2: Điều hướng đến trang tạo cụm GKE

- Ở góc trên bên trái của Console, nhấp vào biểu tượng Menu điều hướng (ba dấu gạch ngang). Cuộn xuống và tìm mục Kubernetes Engine .
- Sau khi tìm được dịch vụ, nhấp vào Kubernetes Engine , sau đó chọn **Clusters**.
- Trên trang Clusters, nhấp **Create** và chọn **Standard: You manage your cluster** → **Configure**

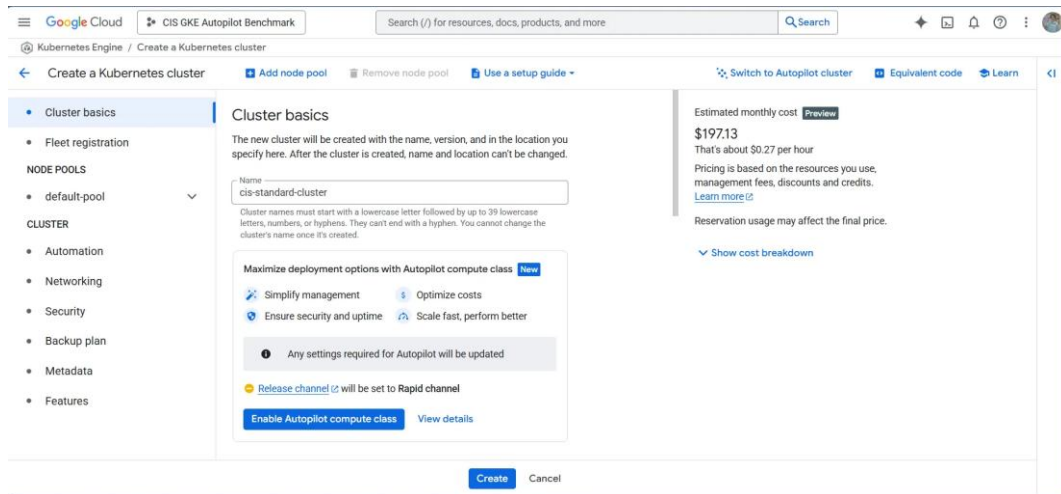
Bước 3: Chọn chế độ Standard và cấu hình cơ bản

- Trên trang “Tạo cụm Kubernetes”, chúng ta sẽ thấy hai tùy chọn chính: Autopilot và Standard . Nhấp vào tùy chọn **Standard** → **Configure**.



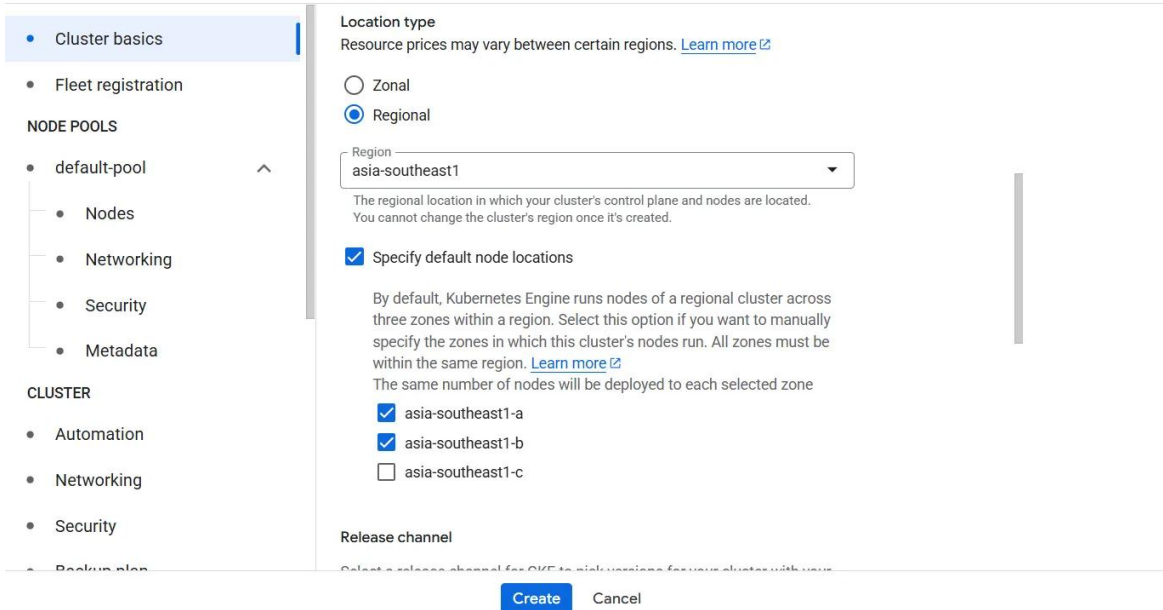
Hình 17. Chọn chế độ cấu hình Standard

- Điền thông tin cơ bản của cụm:
 - **Tên (Name):** cis-standard-cluster



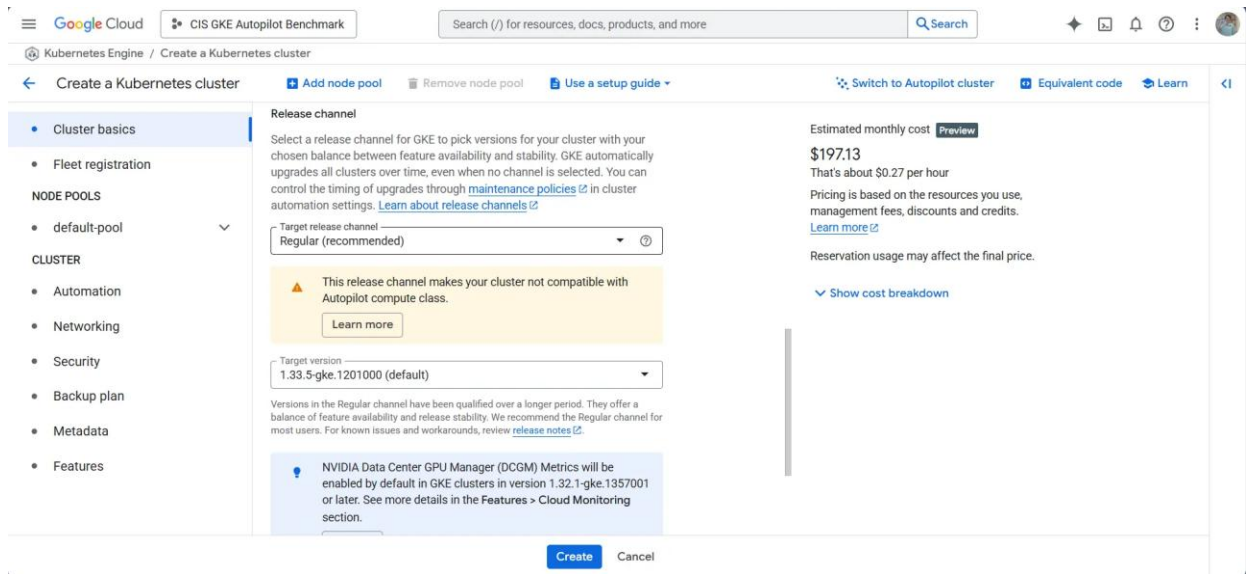
Hình 18. Điền tên cụm

- Cấu hình Loại vị trí (Location type):
 - **Vùng (Zonal):** Chọn tùy chọn này nếu chúng ta muốn cụm của mình triển khai trong một khu vực tính toán (zone) duy nhất. Đây là lựa chọn mặc định và thường đủ cho các trường hợp sử dụng cơ bản.
 - **Khu vực (Regional):** Chọn tùy chọn này nếu chúng ta muốn cụm được phân tán trên nhiều khu vực tính toán (zones) trong một khu vực (region) để tăng tính sẵn sàng và khả năng chịu lỗi → **Chọn Regional**
- **Khu vực/Vùng (Region/Zone):** Chọn khu vực hoặc vùng địa lý gần nhất để giảm độ trễ → Chọn **asia-south1-a**
 - Tùy chọn “**Specify default node locations**” được bật nhằm chỉ định rõ các zone sẽ được sử dụng cho node pool của cụm.



Hình 19. Cấu hình Location Type

- **Phiên bản Kubernetes (Kubernetes version):** Nhóm chọn phiên bản “Release channel” được đề xuất (Regular - Recommended) để xác định nhánh phát hành phiên bản Kubernetes phù hợp với nhu cầu về mức độ ổn định và tốc độ cập nhật.
 - Sau khi chọn kênh Regular, hệ thống tự động gợi ý phiên bản Kubernetes mặc định là *1.33.5-gke.1201000*, đây là phiên bản ổn định thuộc kênh đã chọn.

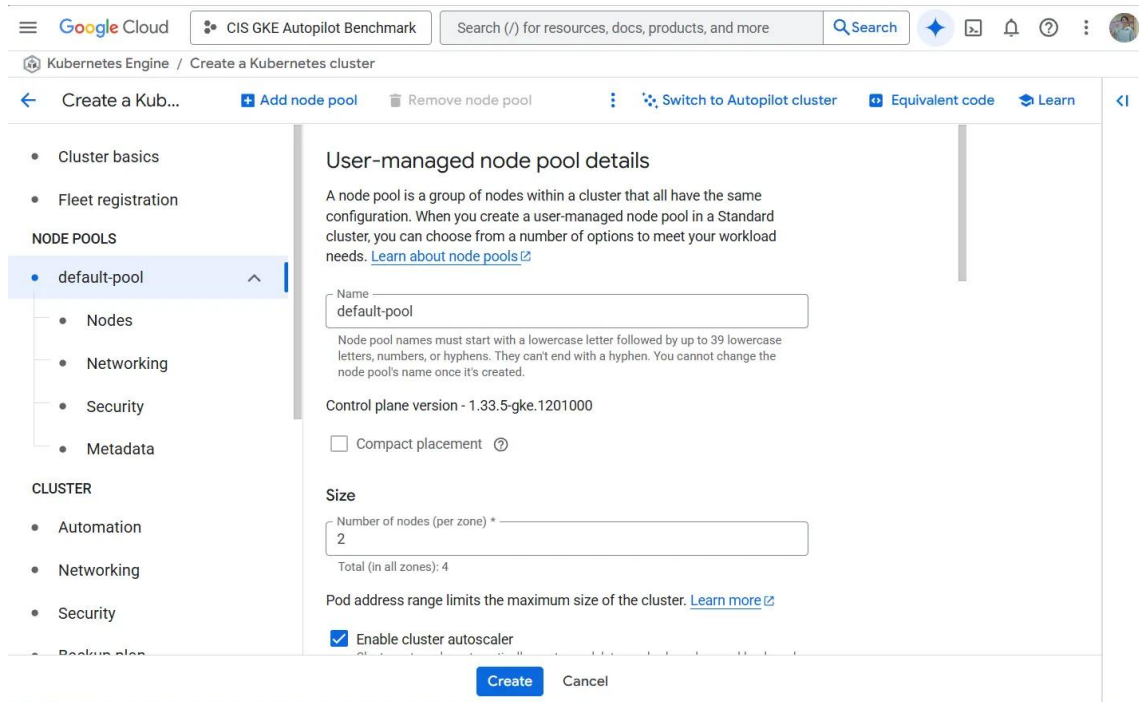


Hình 20. Chọn phiên bản Kubernetes Standard

Bước 4: Cấu hình Node pools

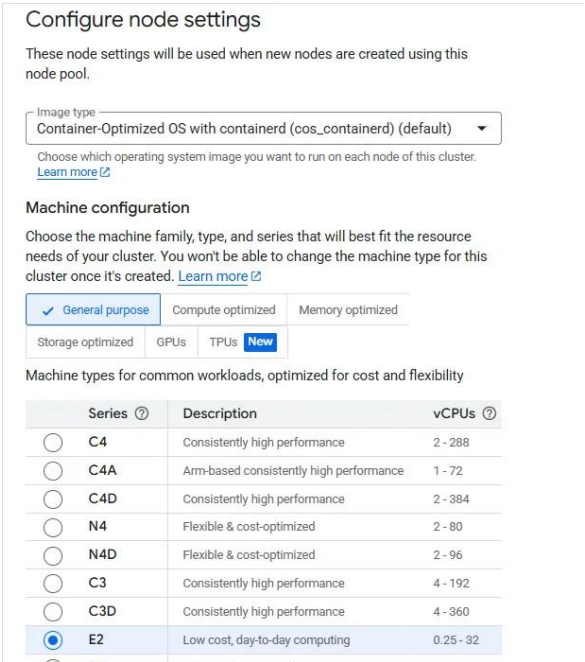
- **Định nghĩa:** Node pools là nơi các máy ảo (VM) sẽ chạy các ứng dụng của chúng ta.
- **Cấu hình Node pool mặc định (Default node pool):**

- **Tên node pool:** *default-pool*
- **Số lượng nút (Number of nodes):** Ở đây nhóm sẽ chọn 2



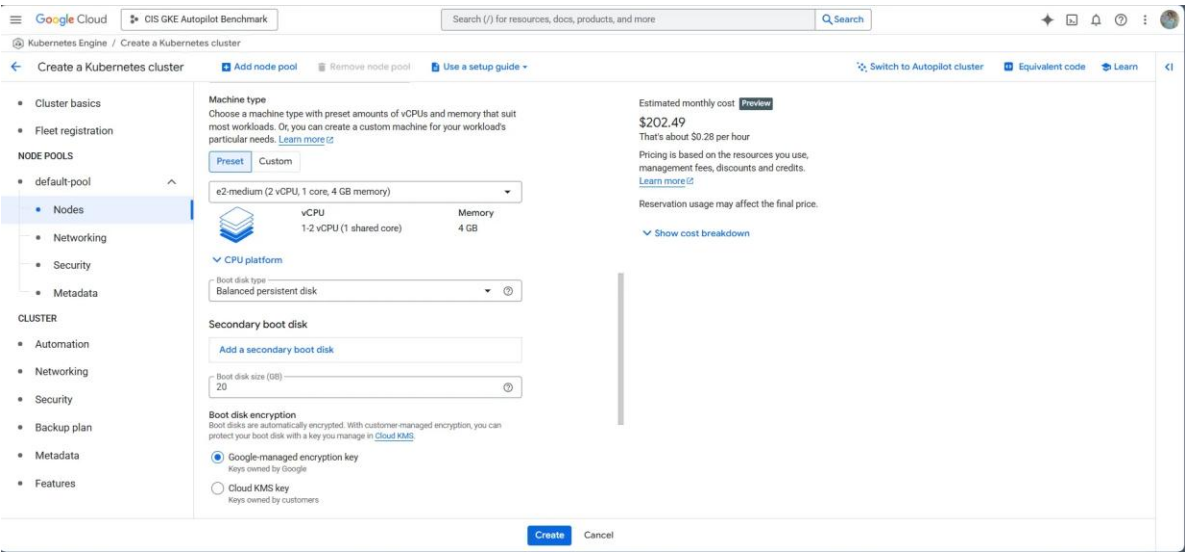
Hình 21. Cấu hình Node pool

- Ở mục Provisioning model, chọn **Standard VMs**, đây là cách Google cho chúng ta chọn loại máy ảo (VM) để chạy node pool trong cluster.
- Cấu hình Nodes:
 - **Image type:** *Container-Optimized OS with containerd (cos_containerd) (default)*
 - Đảm bảo rằng các máy ảo worker node sẽ chạy một hệ điều hành được tối ưu hóa cao về bảo mật và hiệu suất, được thiết kế riêng để chạy các workload vùng chứa trên Kubernetes.
 - Nó sử dụng containerd làm môi trường thời gian chạy vùng chứa, giúp tăng cường hiệu quả và bảo mật. Đây là cấu hình được Google khuyến nghị cho hầu hết các trường hợp sử dụng GKE.



Hình 22. Cấu hình Image type

- Machine configuration:
 - Chọn **General purpose** → **E2 (Low cost, day-to-day computing)**
 - Machine type:** Chọn *e2-medium (2 vCPU, 1 core, 4 GB memory)*
 - Boot disk type (Loại đĩa khởi động):** Chọn *Balanced persistent disk*
 - Boot disk size (Kích thước đĩa khởi động):** Đặt *20GB*
 - Boot disk encryption:** Chọn *Google-managed encryption key (Keys owned by Google)*

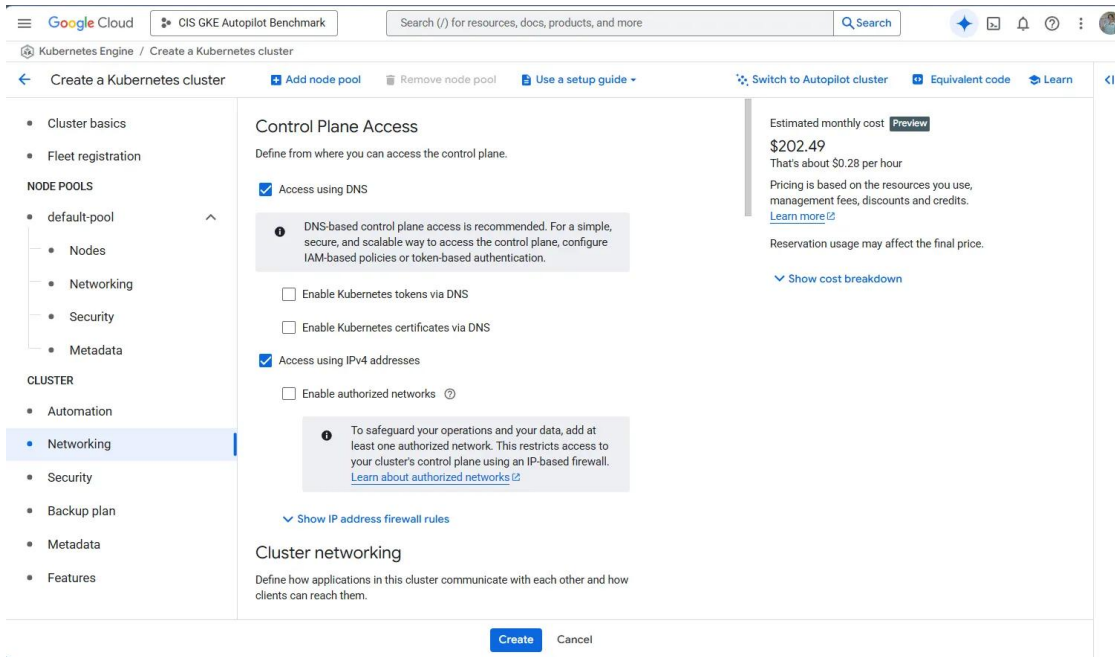


Hình 23. Cấu hình phần cứng

Bước 5: Cấu hình Networking

• Thiết lập quyền truy cập Control Plane

- Bật cơ chế truy cập Control Plane thông qua DNS nhằm đơn giản hóa quá trình kết nối và tăng khả năng mở rộng.
- Cho phép truy cập bằng địa chỉ IPv4, đảm bảo khả năng tương thích với hạ tầng mạng hiện tại.
- Không cấu hình Authorized Networks trong giai đoạn này, cho phép truy cập từ mọi địa chỉ IP phục vụ mục đích thử nghiệm và triển khai ban đầu.



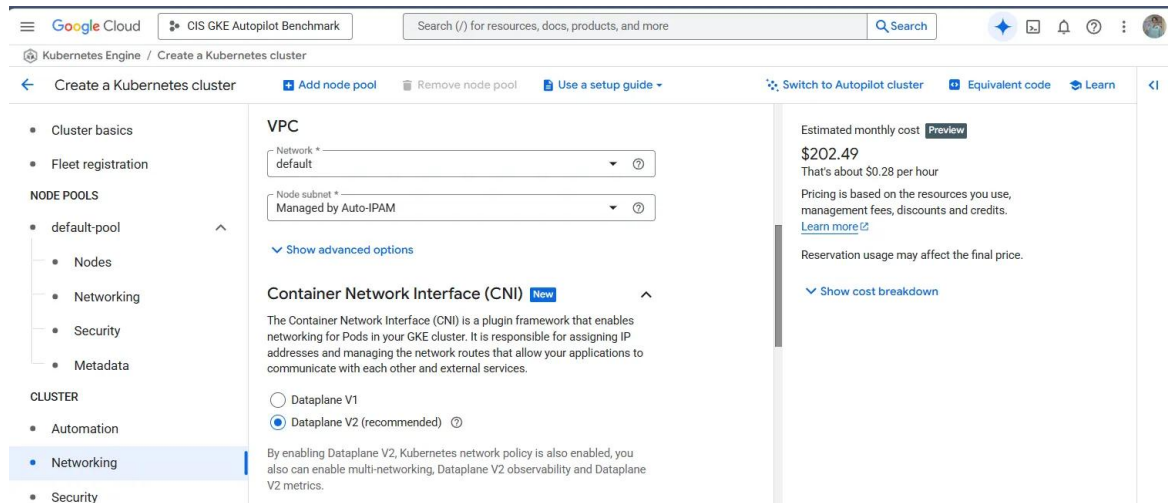
Hình 24. Cấu hình Control Plane Access

• Cấu hình Virtual Private Cloud (VPC)

- Lựa chọn VPC mặc định của dự án Google Cloud để triển khai cụm Kubernetes.
- Sử dụng subnet do hệ thống quản lý, giúp đơn giản hóa việc cấp phát và quản lý địa chỉ IP cho node và pod.

• Thiết lập Container Network Interface (CNI)

- Sử dụng *Dataplane V2 (khuyến nghị)* làm CNI cho cụm Kubernetes.
- Dataplane V2 cho phép kích hoạt Kubernetes Network Policy, hỗ trợ quan sát lưu lượng mạng và cung cấp các chỉ số mạng chi tiết. Đồng thời, nâng cao khả năng bảo mật và kiểm soát lưu lượng giữa các workload trong cluster.



Hình 25. Cấu hình VPC

- **Quản lý địa chỉ IP với Auto-IPAM**

- Kích hoạt **Auto-IPAM** để tự động quản lý và phân bổ không gian địa chỉ IP cho node và pod.
- Giảm thiểu rủi ro xung đột IP và đơn giản hóa quá trình mở rộng cụm trong quá trình vận hành.
- Subnet node được hệ thống tự động quản lý, không yêu cầu cấu hình thủ công.

- **Cấu hình dải địa chỉ cho Pod**

- Thiết lập số lượng Pod tối đa trên mỗi node là 48, phù hợp với khuyến nghị của Auto-IPAM.
- Dải CIDR dành cho Pod được tự động cấp phát và quản lý bởi Auto-IPAM.

- **Cấu hình IP Stack**

- Sử dụng mô hình *IPv4 (single-stack)* cho cụm Kubernetes.
- Đảm bảo khả năng tương thích cao với các dịch vụ và hệ thống mạng hiện tại.

← Create a Kubernetes cluster [Add node pool](#) [Remove node pool](#) [Use a setup guide](#)

- Cluster basics
- Fleet registration
- NODE POOLS
 - default-pool
 - Nodes
 - Networking
 - Security
 - Metadata
- CLUSTER
 - Automation
 - Networking
 - Security
 - Backup plan
 - Metadata
 - Features

By enabling Dataplane V2, Kubernetes network policy is also enabled, you also can enable multi-networking, Dataplane V2 observability and Dataplane V2 metrics.

IP Management

Auto-IPAM [New](#)

Auto-IPAM empowers "growth on demand" with its intelligent, automatic IP allocation, ensuring optimal address space utilization and eliminating wasteful reservations. This streamlined, hands-off approach simplifies cluster scaling and management, particularly for demanding workloads like AI/ML. [Learn more](#)

IPAM settings

☐ Enable Private nodes

☐ Disable default SNAT used for IP masquerading

Pods

Maximum Pods per node:

48 is recommended for Auto-IPAM, but you can change it.
Mask for Pod address range per node: /25

Pod range will be managed by Auto-IPAM. [Learn more](#)

Cluster default Pod address range:

Example: 192.168.0.0/16. [See range size table](#)

IP Stack type

☒ IPv4 (single stack)

☐ IPv4 and IPv6 (dual stack)

Services (not managed by Auto-IPAM)

☒ Use existing range

☐ Create new range for Services

Services secondary CIDR range:

[See range size table](#)

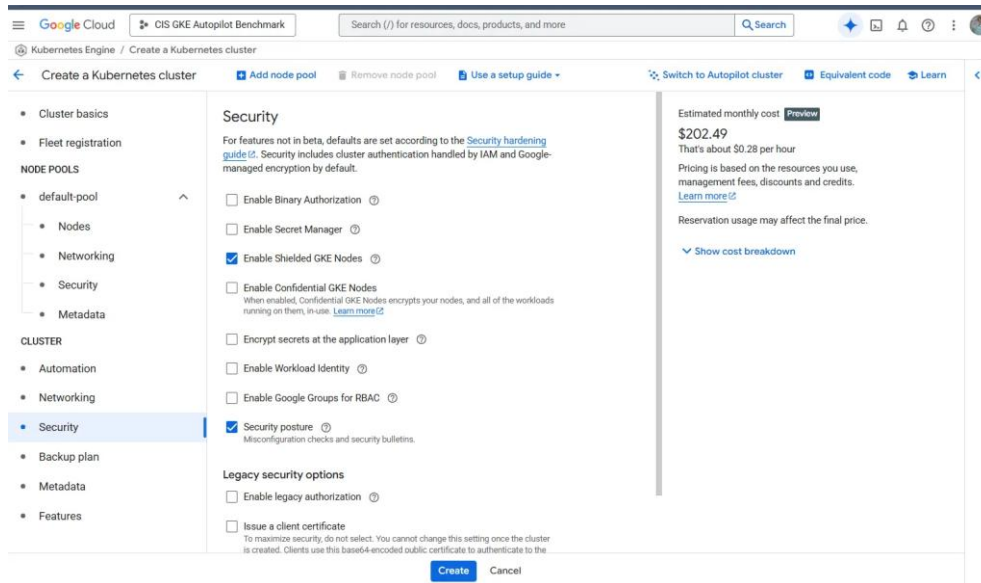
Hình 26. Cấu hình IP Management

• Cấu hình dải địa chỉ cho Service

- Sử dụng dải địa chỉ Service có sẵn do GKE quản lý.
- Dải CIDR cho Service được tách biệt với dải Pod, giúp tránh xung đột địa chỉ và đảm bảo tính ổn định của hệ thống mạng.

Bước 6: Cấu hình nâng cao (Tùy chọn)

- Chúng ta có thể khám phá các tùy chọn nâng cao khác bằng cách nhấp vào các phần tương ứng ở bảng điều khiển bên trái, hoặc nút “Show advanced options” nếu có.
- Bảo mật (Security):** Kích hoạt Workload Identity, Shielded GKE Nodes, ...

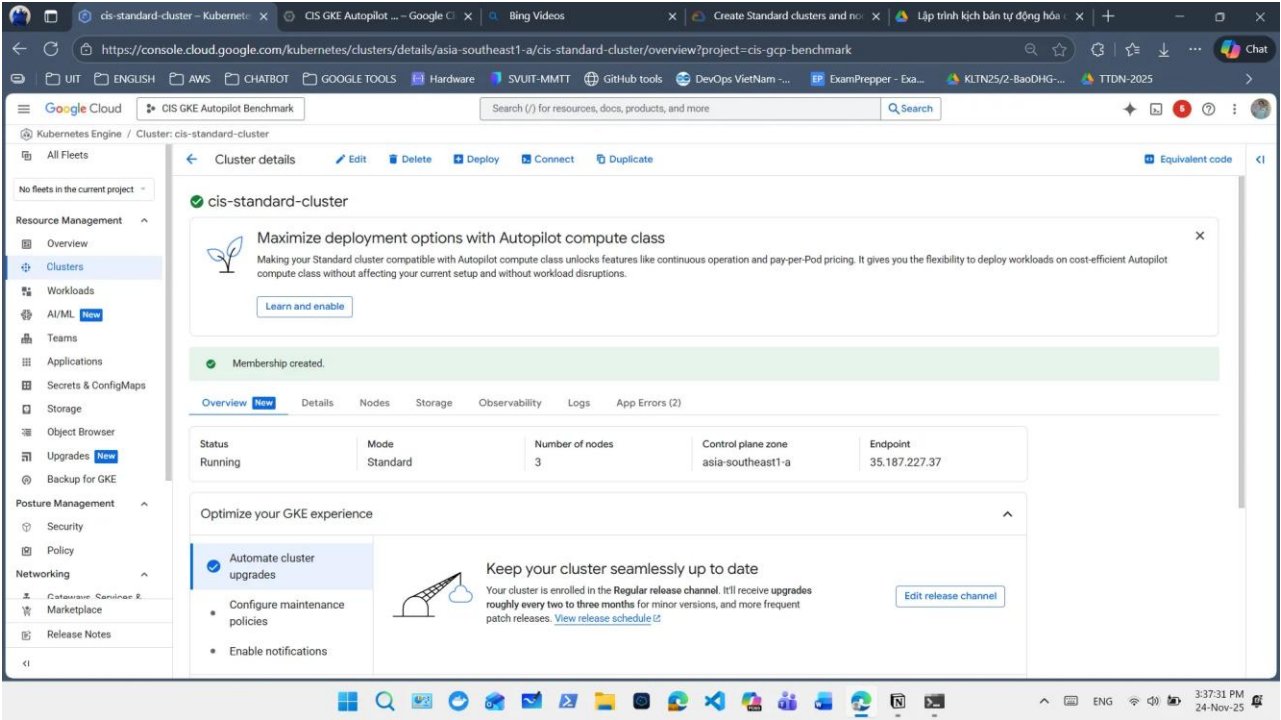


Hình 27. Cấu hình tùy chọn Security

- Tính năng (Features):** Kích hoạt các tính năng như Cloud Monitoring, Cloud Logging, Ingress, Egress, ...
- Metadata (Siêu dữ liệu):** Thêm các nhãn (labels) để quản lý tài nguyên.

Bước 7: Tạo cụm

- Sau khi chúng ta đã cấu hình tất cả các tùy chọn mong muốn, nhấp vào nút **Tạo (Create)** ở cuối trang. Google Cloud sẽ bắt đầu quá trình tạo cụm GKE của nhóm. Quá trình này có thể mất vài phút.
- Sau đó, sẽ được chuyển hướng đến trang “Cụm Kubernetes”, nơi mà chúng ta có thể theo dõi trạng thái tạo cụm. Khi cụm đã sẵn sàng, trạng thái sẽ hiển thị là “Ready”.

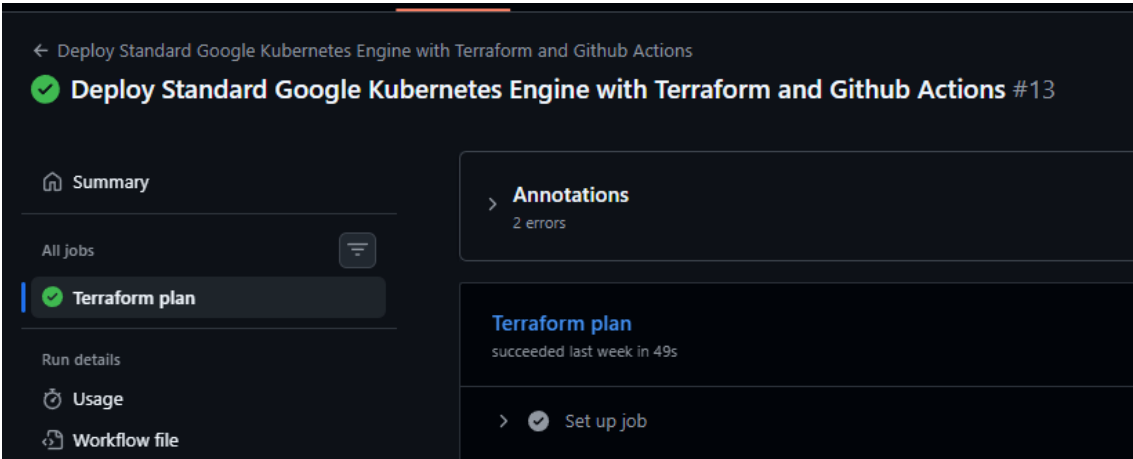


Hình 28. Kết quả khi tạo cụm GKE Standard

3.2.2.2 Phương pháp Automation

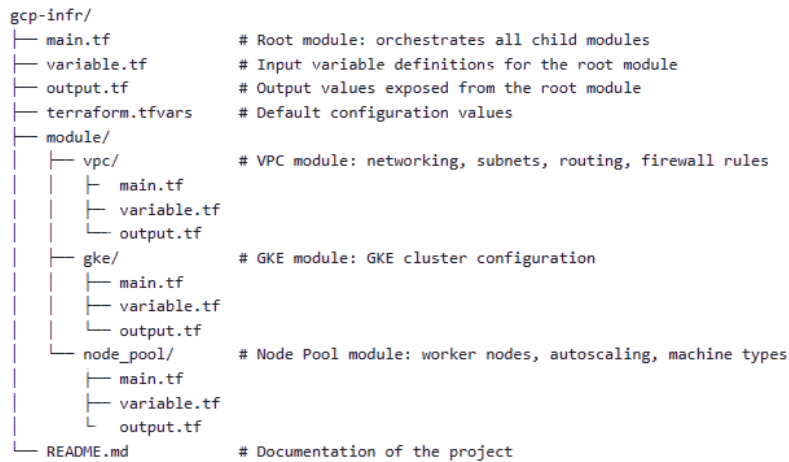
Tạo Github Actions workflow:

- Tương tự như cụm GKE Autopilot, GitHub Actions được sử dụng để tự động hóa quy trình triển khai cụm GKE Standard trên Google Cloud thông qua Terraform. Workflow CI/CD chịu trách nhiệm thực thi các bước khởi tạo và triển khai hạ tầng dựa trên mã nguồn đã được định nghĩa sẵn, đảm bảo tính nhất quán và khả năng tái sử dụng của cấu hình.
- Link cấu hình CI/CD cho cụm Kubernetes Standard: [Link cấu hình](#)



Hình 29. Kết quả workflow

Tạo mã nguồn Terraform để triển khai hạ tầng dưới dạng mã:



Hình 29. Cấu trúc mã nguồn Terraform tạo Kubernetes Standard

- Cấu trúc tổng thể:
 - **Thư mục gốc (gcp-infr/)**: Đóng vai trò là *root module*, chịu trách nhiệm điều phối và liên kết các module con để xây dựng nên toàn bộ hạ tầng.
 - **main.tf**: Định nghĩa các module con (VPC, GKE, Node Pool) và thiết lập mối quan hệ giữa chúng.
 - **variable.tf**: Khai báo các biến đầu vào dùng chung cho toàn bộ hệ thống, giúp cấu hình linh hoạt.
 - **output.tf**: Xuất ra các giá trị quan trọng để dễ theo dõi.
 - **terraform.tfvars**: Cung cấp giá trị mặc định cho các biến, giúp tách cấu hình khỏi logic mã nguồn.
- Module VPC: Quản lý và dựng nền tảng mạng cho toàn bộ hệ thống trên GCP.
 - **main.tf**: Khai báo VPC, subnet, route và firewall rule.
 - **variable.tf**: Định nghĩa các biến như CIDR, region, số lượng subnet.
 - **output.tf**: Xuất ra thông tin VPC và subnet để các module khác sử dụng.
- Module GKE: Tạo và cấu hình cụm Google Kubernetes Engine.
 - **main.tf**: Khai báo cluster GKE, kết nối với VPC đã tạo.
 - **variable.tf**: Cho phép cấu hình tên cluster, version Kubernetes, region/zone.
 - **output.tf**: Xuất thông tin về endpoint, tên của cluster phục vụ cho quá trình triển khai ứng dụng.
- Module Node Pool: Cấu hình các node pool cho GKE cluster.

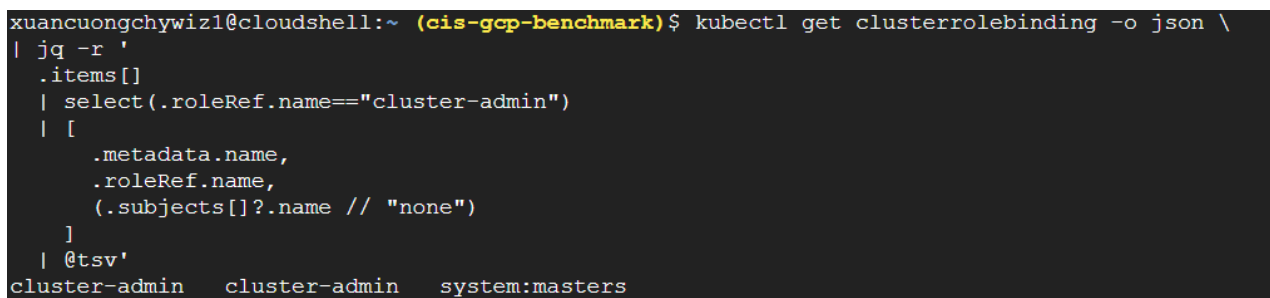
- **main.tf**: Khai báo node pool, machine type, autoscaling.
- **variable.tf**: Tùy chỉnh số lượng node, giới hạn scale, loại máy.
- **output.tf**: Xuất thông tin node pool để theo dõi và quản lý.

3.3. Triển khai Manual

3.3.1: Đảm bảo vai trò cluster-admin chỉ được sử dụng khi cần thiết

Audit:

- Mục tiêu : Xác định các user hoặc service account đang được gán vai trò *cluster-admin*.
- Kiểm tra các *ClusterRoleBinding* liên kết tới *cluster-admin*.
- Đánh giá tính cần thiết của từng đối tượng được gán quyền.



```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl get clusterrolebinding -o json \
| jq -r '
.items[]
| select(.roleRef.name=="cluster-admin")
| [
  .metadata.name,
  .roleRef.name,
  (.subjects[]?.name // "none")
]
| @tsv'
cluster-admin    cluster-admin    system:masters
```

Hình 30. Kiểm tra vai trò cluster-admin

→ *cluster-admin* thuộc nhóm system:masters là cấu hình mặc định → Không có clusterrole nào vi phạm

Remediation:

- Mục tiêu: Loại bỏ việc sử dụng *cluster-admin* không cần thiết và áp dụng nguyên tắc least privilege.
- Gỡ bỏ *cluster-admin* khỏi user/service account không cần thiết

→ Sử dụng lệnh *delete kubectl delete clusterrolebinding*

Autopilot: Mặc định autopilot sẽ chặn các *non-admin user bind cluster-admin*

```

<none>
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark) $ kubectl auth can-i "*" "
*" --as=system:service:default:cis-test-sa
no
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark) $ gcloud container cluster
s describe autopilot-cluster --region asia-southeast1 --format="value(a
utopilot.enabled)"
True

```

Hình 31. Autopilot mặc định chặn non-admin user

3.3.2: Giảm thiểu việc sử dụng ký tự đại diện (wildcard) trong Roles và ClusterRoles

Audit:

- Mục tiêu :
 - Phát hiện các Role và ClusterRole sử dụng *wildcard permissions*.
 - Kiểm tra verbs: ["*"]
 - Kiểm tra resources: ["*"]
 - Kiểm tra apiGroups: ["*"]

```

Bash ▾
kubectl get roles --all-namespaces -o json | jq -r '
  .items[]
  | select(
    (.rules[]?.verbs[]? == "*") or
    (.rules[]?.apiGroups[]? == "*") or
    (.rules[]?.resources[]? == "*") or
    (.rules[]?.resourceNames[]? == "*")
  )
  | .metadata.namespace + "/" + .metadata.name
'

```

Hình 32. Kiểm tra quyền wildcard

Remediation:

- Mục tiêu:
 - Giới hạn quyền truy cập đúng theo nhu cầu thực tế.
 - Lấy role vi phạm bằng `kubectl edit role <role-name> -n <name-space>`
 - Chỉnh sửa quyền wildcard bằng các quyền cụ thể như (get,list,...)

Autopilot: Mặc định sẽ chặn các *non-admin* tạo các *role-binding wildcard*

```

xuancuongchywiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl auth can-i get nodes --as=system:serviceaccount:default:cis-wildcard-bind
Warning: resource 'nodes' is not namespace scoped
no
xuancuongchywiz1@cloudshell:~ (cis-gcp-benchmark)$ gcloud container clusters describe autopilot-cluster --region asia-southeast1 --format="value(autopilot.enabled)"
True

```

Hình 33. Autopilot chặn wildcard không phù hợp

3.3.3: Giới hạn việc sử dụng quyền Bind, Impersonate và Escalate trong Kubernetes cluster

Audit:

- Mục tiêu:
 - Phát hiện các role có quyền *escalate*.
 - Role có quyền *bind*
 - Role có quyền *impersonate*

```

xuancuongchywiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl get clusterrolebinding
-o yaml | grep -A5 -E "bind|impersonate|escalate"
  name: anet-operator-role-binding
  resourceVersion: "1765380351245839014"
  uid: c27ef8ba-aa08-4591-bc50-a0523af1b564
  roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
  ---
  {"apiVersion":"rbac.authorization.k8s.io/v1","kind":"ClusterRoleBinding",
  "metadata":{"annotations":{},"name":"cis-escalate-bind-impersonate-binding"},"rol
  eRef":{"apiGroup":"rbac.authorization.k8s.io","kind":"ClusterRole","name":"cis-es
  calate-bind-impersonate-test"},"subjects":[{"kind":"ServiceAccount","name":"cis-t
  est-sa","namespace":"default"}]}

```

Hình 34. Kiểm tra quyền Bind, Impersonate và Escalate

Remediation

- Mục tiêu:
 - Ngăn chặn leo thang đặc quyền.
 - Gỡ quyền *escalate*, *bind*, *impersonate* nếu không cần

```

xuancuongchywiz1@cloudshell:~ (cis-gcp-benchmark)$ cat sensitive.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: sensitive
rules:
- apiGroups: ["rbac.authorization.k8s.io"]
  resources: ["roles", "clusterroles", "rolebindings", "clusterrolebindings"]
  verbs: ["bind", "impersonate", "escalate"]
xuancuongchywiz1@cloudshell:~ (cis-gcp-benchmark)$ 

```

Hình 35. Tạo quyền độc hại

```

xuancuongchywiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl delete clusterrole sensitive
clusterrole.rbac.authorization.k8s.io "sensitive" deleted

```

Hình 36. Xóa quyền vừa tạo

Autopilot: Mặc định thì autopilot sẽ chặn các *non-admin* có các quyền *bind/escalate/imperonate*

```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl auth can-i create clusterrolebinding --as=system:serviceaccount:default:cis-test-sa
Warning: resource 'clusterrolebindings' is not namespace scoped in group 'rbac.authorization.k8s.io'
no
```

Hình 37. Autopilot tự động chặn

```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl auth can-i escalate clusterrolebinding --as=system:serviceaccount:default:cis-test-sa
Warning: resource 'clusterrolebindings' is not namespace scoped in group 'rbac.authorization.k8s.io'
no
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl auth can-i impersonate clusterrolebinding --as=system:serviceaccount:default:cis-test-sa
Warning: resource 'clusterrolebindings' is not namespace scoped in group 'rbac.authorization.k8s.io'
no
```

Hình 38. Autopilot chặn tạo quyền escalate và impersonate

3.3.4: Đảm bảo cụm áp dụng và thực thi hồ sơ Pod Security Standard Baseline cho tất cả các namespace

Audit:

- **Mục tiêu:** Xác định các namespace chưa được gán *PSS Baseline*.

```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl get ns --show-labels
NAME                STATUS   AGE   LABELS
default             Active  42h   kubernetes.io/metadata.name=default
gke-managed-cim     Active  42h   addonmanager.kubernetes.io/mode=Reconcile,kubernetes.io/metadata.name=gke-managed-cim
gke-managed-dpv2-observability Active  42h   addonmanager.kubernetes.io/mode=Reconcile,kubernetes.io/metadata.name=gke-managed-dpv2-observability
gke-managed-system Active  42h   addonmanager.kubernetes.io/mode=Reconcile,kubernetes.io/metadata.name=gke-managed-system
gke-managed-volumepopulator Active  42h   addonmanager.kubernetes.io/mode=Reconcile,k8s-app=gke-volume-populator,kubernetes.io/metadata.name=gke-managed-volumepopulator
gmp-public          Active  42h   addonmanager.kubernetes.io/mode=Reconcile,kubernetes.io/metadata.name=gmp-public
gmp-system           Active  42h   addonmanager.kubernetes.io/mode=Reconcile,kubernetes.io/metadata.name=gmp-system
kube-node-lease      Active  42h   kubernetes.io/metadata.name=kube-node-lease
kube-public          Active  42h   kubernetes.io/metadata.name=kube-public
kube-system          Active  42h   kubernetes.io/metadata.name=kube-system
```

Hình 39. Kiểm tra namespace nào chưa gán PSS

Remendiation:

- **Mục tiêu:** Áp dụng *PSS Baseline* cho namespace.

```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ for ns in $(kubectl get ns --no-headers | awk '{print $1}'); do
  kubectl label ns $ns \
    pod-security.kubernetes.io/enforce=baseline \
    pod-security.kubernetes.io/audit=baseline \
    pod-security.kubernetes.io/warn=baseline \
    --overwrite;
done
namespace/default labeled
namespace/dev-environment not labeled
namespace/gke-managed-cim labeled
namespace/gke-managed-dpv2-observability labeled
namespace/gke-managed-system labeled
namespace/gke-managed-volumepopulator labeled
namespace/gmp-public labeled
namespace/gmp-system labeled
namespace/kube-node-lease labeled
namespace/kube-public labeled
namespace/kube-system labeled
```

Hình 40. Áp dụng PSS

Autopilot: Các pod sẽ được bật sẵn baseline

```
xuancuongchwyiz1@cloudshell:~ (cis-gcp-benchmark)$ nano privileged.yaml
xuancuongchwyiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl apply -f privileged.yaml
Warning: autopilot-default-resources-mutator:Autopilot updated Pod default/privileged-pod: defaulted unspecified 'cpu' resource for containers [test] (see http://g.co/gke/autopilot-defaults).
Error from server (GKE Warden constraints violations): error when creating "privileged.yaml": admission webhook "warden-validating.common-webhooks.networking.gke.io" denied the request: GKE Warden rejected the request because it violates one or more constraints.
Violations details: {"[denied by autogke-disallow-privilege]":["container test is privileged; not allowed in Autopilot"]}
Requested by user: 'xuancuongchwyiz1@gmail.com', groups: 'system:authenticated'.
xuancuongchwyiz1@cloudshell:~ (cis-gcp-benchmark)$ gcloud container clusters describe autopilot-cluster --region asia-southeast1 --format="value(autopilot.enabled)"
True
```

Hình 41. Autopilot bật sẵn PSS

3.3.5: Tạo ranh giới quản trị giữa các tài nguyên bằng cách sử dụng namespaces

Audit:

- Mục tiêu:
 - Xác định workload không được phân tách namespace hợp lý.
 - Ứng dụng không chạy trong namespace riêng.
 - Dùng chung namespace cho nhiều môi trường (dev/prod).
 - RBAC áp dụng cluster-wide thay vì namespace-level.

```
xuancuongchwyiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	violate-76f979cb59-tfzvz	1/1	Running	0	7m39s
dev	random-pod	1/1	Running	0	5m59s
gke-managed-cim	kube-state-metrics-0	2/2	Running	0	45h
gke-managed-dpv2-observability	hubble-relay-5d46689f4b-2lx5l	3/3	Running	0	45h
gmp-system	collector-g6jxj	2/2	Running	0	45h
gmp-system	collector-k5fmw	2/2	Running	0	45h
gmp-system	collector-xfhnk	2/2	Running	0	45h
gmp-system	gmp-operator-749659cdb8-pdzkk	1/1	Running	1 (45h ago)	45h
kube-system	anetd-5rspr	3/3	Running	0	45h
kube-system	anetd-hpkdz	3/3	Running	0	45h
kube-system	anetd-wmhcq	3/3	Running	0	45h
kube-system	antrea-controller-horizontal-autoscaler-5746cc86d8-pp59d	1/1	Running	0	45h
kube-system	event-exporter-gke-5d586d6664-8szdd	2/2	Running	0	45h

Hình 42. Kiểm tra phân tách namespace

Remediation:

- Tạo namespace riêng cho từng ứng dụng / môi trường.
- Áp dụng RBAC theo namespace.
- Kết hợp NetworkPolicy để kiểm soát traffic liên namespace.

```
xuancuongchwyiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl apply -f dev_role_bindings.yaml
error: Unexpected args: [dev_role_bindings.yaml]
See 'kubectl apply -h' for help and examples
xuancuongchwyiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl apply -f dev_role_bindings.yaml
rolebinding.rbac.authorization.k8s.io/dev-access created
xuancuongchwyiz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl apply -f dev_role.yaml
role.rbac.authorization.k8s.io/dev-role created
```

Hình 43. Tạo namespace cho dev và áp dụng

Autopilot: Mỗi role được tạo sẽ bị giới hạn quyền trong namespace tương ứng.

```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl get sa cis-test-sa -n default
NAME          SECRETS  AGE
cis-test-sa    0        48m
```

Hình 44. Autopilot tự động giới hạn quyền namespace

```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ kubectl auth can-i get pods --namespace=dev --as=system:serviceaccount:default:cis-test-sa no
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ gcloud container clusters describe autopilot-cluster --region asia-southeast1 --format="value(autopilot.enabled)" true
```

Hình 45. Autopilot xóa quyền namespace không phù hợp

3.3.6 Bật VPC Flow Logs và khả năng quan sát lưu lượng nội bộ giữa các node

Audit:

- Mục tiêu: Xác minh VPC Flow Logs và Intranode Visibility đã được bật hay chưa.

```
creationTimestamp: '2025-11-24T00:29:33.440-08:00'
description: auto-created subnetwork for cluster "cis-standard-cluster"
fingerprint: MWdlqAmxNU=
gatewayAddress: 192.168.0.1
id: '9203087892942530946'
ipCidrRange: 192.168.0.0/24
kind: compute#subnetwork
name: gke-cis-standard-cluster-subnet-e65a302d
network: https://www.googleapis.com/compute/v1/projects/cis-gcp-benchmark/global/networks/default
privateIpGoogleAccess: true
privateIpv6GoogleAccess: DISABLE_GOOGLE_ACCESS
purpose: PRIVATE
region: https://www.googleapis.com/compute/v1/projects/cis-gcp-benchmark/regions/asia-southeast1
reservedInternalRange: https://networkconnectivity.googleapis.com/v1/projects/cis-gcp-benchmark/locations/global/internalRanges/gke-cis-standard-cluster-subnet-e65a302d
secondaryIpRanges:
- ipCidrRange: 192.168.64.0/18
  rangeName: gke-cis-standard-cluster-pods-e65a302d
  reservedInternalRange: https://networkconnectivity.googleapis.com/v1/projects/cis-gcp-benchmark/locations/global/internalRanges/gke-cis-standard-cluster-pods-e65a302d
selfLink: https://www.googleapis.com/compute/v1/projects/cis-gcp-benchmark/regions/asia-southeast1/subnetworks/gke-cis-standard-cluster-subnet-e65a302d
stackType: IPV4_ONLY
```

Hình 46. Kiểm tra VPC flow logs

```
xuancuongchyz1@cloudshell:~ (cis-gcp-benchmark)$ gcloud container clusters describe cis-standard-cluster \
> --region asia-southeast1-a | grep -A2 intraNodeVisibility
```

Hình 47. Kiểm tra Intranode Visibility

Remediation :

- Bật *VPC Flow Logs* cho subnet.
- Bật *Intranode Visibility* khi tạo cluster.

Autopilot: Tự động bật *VPC Flow Logs* và *Intranode Visibility*.

```
xuancuongchyz1@cloudshell:~/scripts/scripts (cis-gcp-benchmark)$ gcloud compute networks subnets describe gke-subnet-v2 --region asia-southeast1 --format="value(enableFlow Logs)"
True
xuancuongchyz1@cloudshell:~/scripts/scripts (cis-gcp-benchmark)$
```

Hình 48. Autopilot tự động bật VPC Flow Logs

```
xuancuongchyz1@cloudshell:~/scripts/scripts (cis-gcp-benchmark)$ gcloud container clusters describe cis-standard-cluster --region asia-southeast1-a --format="value(network Config.enableIntraNodeVisibility)"
True
xuancuongchyz1@cloudshell:~/scripts/scripts (cis-gcp-benchmark)$
```

Hình 48. Autopilot tự động bật Intranode Visibility

3.4 Triển khai Automation

Phương pháp: Để đảm bảo tính nhất quán, khả năng lặp lại và giảm thiểu sai sót trong quá trình đánh giá, các bước audit thủ công (manual audit) được thực hiện lại thông qua các script tự động. Các script này có nhiệm vụ thu thập cấu hình thực tế của cluster Kubernetes (GKE Standard) dựa trên các tiêu chí của từng control trong CIS Benchmark.

Quy trình Audit và Đánh giá PASS/FAIL :

- Quá trình audit được thực hiện bằng cách sử dụng các *script kiểm tra* cấu hình áp dụng cho GKE Standard cluster để thu thập trạng thái cấu hình thực tế.
- Kết quả đầu ra của script sau đó được đối chiếu với các tiêu chí bảo mật tương ứng trong môi trường GKE Autopilot, có xét đến các cơ chế bảo mật được Autopilot enforce mặc định.
- Việc đánh giá tuân thủ được thực hiện theo nguyên tắc:
 - **PASS:** Cấu hình đáp ứng yêu cầu bảo mật hoặc được Autopilot kiểm soát mặc định
 - **FAIL:** Cấu hình chưa đáp ứng yêu cầu hoặc cần thực hiện remediation bổ sung
- Kết quả PASS/FAIL được ghi nhận riêng cho từng control nhằm phục vụ đánh giá và khắc phục sau audit.

3.4.1 Kiểm tra quyền non-admin clusterrole-binding

```

echo ""
echo "===== GKE CIS 4.1.1 Automation ====="

echo "=== Detect unauthorized cluster-admin bindings ==="

VIOLATIONS=$(kubectl get clusterrolebindings -o json \
| jq -r '.items[]' \
| select(.roleRef.name=="cluster-admin") \
| {name: .metadata.name, subjects: [.subjects[].name]} \
| select(.subjects[] != "system:masters") \
| .name'
)

if [ -z "$VIOLATIONS" ]; then
  echo "[PASS] No unauthorized cluster-admin bindings detected."
  exit 0
fi

echo "[FAIL] Unauthorized cluster-admin bindings found:"
echo "$VIOLATIONS"
exit 1

```

Hình 49. Bash script cho Kiểm tra quyền non-admin

- Chạy lệnh `kubectl get clusterrolebindings` tương tự như ở manual, kết quả trả về biến *Violations* (đã loại trừ các rolebinding mặc định)
- Đánh giá **PASS/FAIL** dựa trên có violations trả về hay không.

3.4.2. Pod Security Standard (PSS):

```
while read -r ns_name; do
    if is_system_ns "$ns_name"; then
        continue
    fi

    enforce_level=$(kubectl get ns "$ns_name" -o json | jq -r '.metadata.labels["pod-security.kubernetes.io/enforce"] // "none"')

    if [[ "$enforce_level" != "baseline" && "$enforce_level" != "restricted" ]]; then
        HAS_VIOLATION=true
        VIOLATIONS+=("$ns_name")
    fi
done < <(kubectl get ns -o json | jq -r '.items[].metadata.name')

echo ""

if [[ "$HAS_VIOLATION" = false ]]; then
    echo "[PASS] All user namespaces enforce Pod Security Baseline or higher."
    exit 0
else
    echo "[FAIL] Pod Security violations found: ${#VIOLATIONS[@]} namespaces - ${VIOLATIONS[*]}"
    exit 1
fi
```

Hình 50. Bash script cho PSS

- Kiểm tra trong labels các namespace đã có *enforce=baseline* chưa, nếu chưa có thì các namespace sẽ được thêm vào **violations**.
- Dựa trên số lượng violation và trạng thái HAS_VIOLATION để trả về **PASS/FAIL**, nếu **FAIL** sẽ trả thêm danh sách các violations.

3.4.3 General Policy: Ranh giới giữa namespace

```
NP_COUNT=$(kubectl get networkpolicy -n "$ns" --no-headers 2>/dev/null | wc -l)
if [[ "$NP_COUNT" -gt 0 ]]; then
    echo " NetworkPolicy: PASS ($NP_COUNT policies found)"
    PASS_COUNT=$((PASS_COUNT + 1))
else
    echo " NetworkPolicy: WARN (no policies found)"
    WARN_COUNT=$((WARN_COUNT + 1))
fi

# Check ResourceQuota
RQ_COUNT=$(kubectl get resourcequota -n "$ns" --no-headers 2>/dev/null | wc -l)
if [[ "$RQ_COUNT" -gt 0 ]]; then
    echo " ResourceQuota: PASS ($RQ_COUNT quotas found)"
    PASS_COUNT=$((PASS_COUNT + 1))
else
    echo " ResourceQuota: WARN (no quotas found)"
    WARN_COUNT=$((WARN_COUNT + 1))
fi

# Check RBAC cross-namespace: roles granting access to other namespaces
CROSS_ROLE_COUNT=$(kubectl get roles,clusterroles -n "$ns" -o json 2>/dev/null | jq '[.items[] | .rules[]? | select(.resources? != null) | select(.resources[]? != null)]' | wc -l)
if [[ "$CROSS_ROLE_COUNT" -gt 0 ]]; then
    echo " RBAC cross-namespace: WARN ($CROSS_ROLE_COUNT potentially over-permissive rules)"
    WARN_COUNT=$((WARN_COUNT + 1))
else
    echo " RBAC cross-namespace: PASS"
    PASS_COUNT=$((PASS_COUNT + 1))
fi
```

Hình 51. Bash script cho namespace

- Sử dụng *NP-count* là kết quả các namespace với giá trị networkpolicy được trả về, nếu có thì sẽ được được +1 vào biến PASS-COUNT.
- Sử dụng *RQ-COUNT* để check resource_quota, nếu có quota thì sẽ thêm vào WARN để cảnh báo,

nếu không thì thêm vào `pass_count`.

- Sử dụng *CROSS-ROLE-COUNT* để kiểm tra xem có role nào mà quyền của nó nằm nhiều hơn trong 1 namespace hay không, nếu có thì thêm vào WARN, nếu không thì PASS.
- Sau khi đã tổng hợp được số lượng WARN và PASS thì sẽ đánh giá PASS/WARN.

3.4.4 Cluster Networking: Bật VPC flowlogs và IntraNodeVisibility

```
FLOW_LOGS=$(gcloud compute networks subnets describe "$SUBNET" \
  --region "$REGION" \
  --format="value(enableFlowLogs)" 2>/dev/null)

FLOW_STATUS="FAIL"
if [[ "$FLOW_LOGS" == "True" ]]; then
  echo "[PASS] VPC Flow Logs are ENABLED ✓"
  FLOW_STATUS="PASS"
else
  echo "[FAIL] VPC Flow Logs are DISABLED ✗"
fi

echo ""
```

Hình 52. Bash script cho VPC Flow logs

```
INTRA=$(gcloud container clusters describe "$CLUSTER_NAME" \
  --region "$ZONE" \
  --format="value(networkConfig.enableIntraNodeVisibility)" 2>/dev/null)

INTRA_STATUS="FAIL"
if [[ "$INTRA" == "True" ]]; then
  echo "[PASS] Intra-node Visibility is ENABLED ✓"
  INTRA_STATUS="PASS"
else
  echo "[FAIL] Intra-node Visibility is DISABLED ✗"
fi
```

Hình 53. Bash script cho IntraNodeVisibility

- Kiểm tra tình trạng của VPC flowlogs và IntraNodeVisibility dựa trên kết quả trả về của *FLOW-STATUS* và *INTRANODE-STATUS*
- Sau đó dựa vào kết quả của hai trạng thái này để đánh giá PASS/FAIL, nếu 1 trong 2 không FAIL thì cả mục đánh giá sẽ fail, đồng thời cũng sẽ show ra trạng thái nào FAIL.

Kết quả của việc chạy scripts:

```

✓ cis_4.1.1.sh (RBAC) - Cluster_admin_role_check
✓ cis_4.1.3.sh (RBAC) - Clusterrole and role wildcard check
✓ cis_4.1.7.sh (RBAC) - Bind,Impersonate,Escalate role and clusterrole check
✓ cis_4.2.1.sh (PSS) - Baseline enforcement check
✓ cis_4.6.1.sh (GeneralPolicy) - Namespace Boundary enforcement check
✓ cis_5.4.1.sh (Networking) - VPC flowlogs and IntraNodeVisibility check

=====

Do you want to run all checks? (yes/no): yes

👉Chạy tất cả checks...

Chạy: cis_4.1.1.sh (RBAC) - Cluster_admin_role_check
✓cis_4.1.1.sh đã chạy xong (Status: PASS)
Chạy: cis_4.1.3.sh (RBAC) - Clusterrole and role wildcard check
✓cis_4.1.3.sh đã chạy xong (Status: PASS)
Chạy: cis_4.1.7.sh (RBAC) - Bind,Impersonate,Escalate role and clusterrole check
✓cis_4.1.7.sh đã chạy xong (Status: FAIL)
Chạy: cis_4.2.1.sh (PSS) - Baseline enforcement check
✓cis_4.2.1.sh đã chạy xong (Status: PASS)
Chạy: cis_4.6.1.sh (GeneralPolicy) - Namespace Boundary enforcement check
✓cis_4.6.1.sh đã chạy xong (Status: PASS)
Chạy: cis_5.4.1.sh (Networking) - VPC flowlogs and IntraNodeVisibility check
✓cis_5.4.1.sh đã chạy xong (Status: PASS)

✓Log saved to: log.txt
Report generated: report.html
=== Tất cả checks đã hoàn thành ===

```

Hình 54. Chạy script tổng hợp

File log.txt:

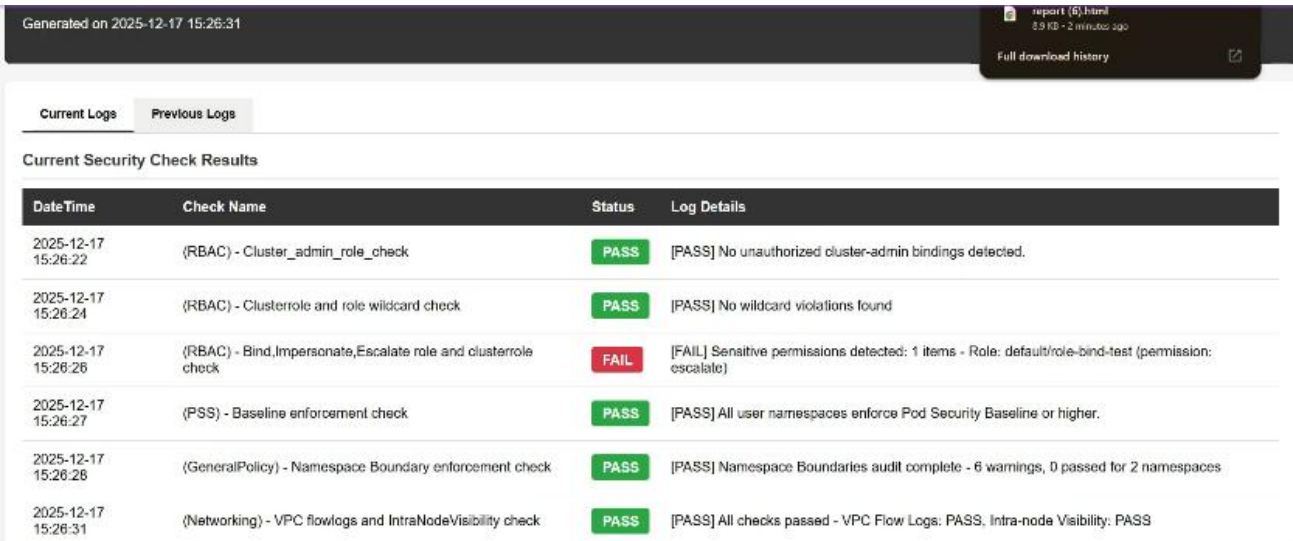
```

1  [
2    {"datetime": "2024-01-15 10:30:45", "name": "cis_5.4.1.sh", "status": "PASS", "log": "Security check passed"},
3    {"datetime": "2024-01-15 10:31:12", "name": "cis_4.1.1.sh", "status": "PASS", "log": "Configuration verified"},
4    {"datetime": "2024-01-15 10:31:45", "name": "cis_4.1.3.sh", "status": "FAIL", "log": "Failed: Invalid permissions detected"},
5    {"datetime": "2024-01-15 10:32:20", "name": "cis_4.1.7.sh", "status": "PASS", "log": "Audit logging enabled"},
6    {"datetime": "2024-01-15 10:33:05", "name": "cis_4.2.1.sh", "status": "PASS", "log": "SSH configuration compliant"},
7    {"datetime": "2024-01-15 10:33:50", "name": "cis_4.6.1.sh", "status": "PASS", "log": "Access control verified"},
8    {"datetime": "2024-01-15 10:34:15", "name": "cis_5.4.1.sh", "status": "PASS", "log": "Security check passed"},
9    {"datetime": "2024-01-15 10:34:45", "name": "cis_4.1.1.sh", "status": "PASS", "log": "Configuration verified"},
10   {"datetime": "2024-01-15 10:35:15", "name": "cis_4.1.3.sh", "status": "FAIL", "log": "Failed: Invalid permissions detected"},
11   {"datetime": "2024-01-15 10:35:50", "name": "cis_4.1.1.sh", "status": "PASS", "log": "Configuration verified"}
12 ]

```

Hình 55. File log.txt

Hiển thị HTML để dễ quan sát :



Hình 56. File log bằng HTML

4. ĐÁNH GIÁ VÀ KẾT LUẬN

4.1 . Bảng so sánh

CIS Rule	Standard GKE Cluster	Autopilot GKE Cluster
CIS 4.1.1 – Cluster-admin	PASS: Thực hiện audit ClusterRole và loại bỏ các quyền cluster-admin không cần thiết	PASS: Quyền cluster-admin được kiểm soát và enforced bởi hệ thống
CIS 4.1.3 – Wildcard Permissions	PASS: Audit các ClusterRole/Role và loại bỏ quyền wildcard nhạy cảm	PASS: Hệ thống tự động enforce và hạn chế wildcard permissions
CIS 4.1.7 – Impersonate / Bind / Escalate	PASS: Audit RBAC và loại bỏ các quyền impersonate, bind, escalate không cần thiết	PASS: Các quyền nhạy cảm được enforce và kiểm soát mặc định
CIS 4.2.1 – Pod Security Standards (Baseline)	PASS: Audit và cấu hình thủ công Pod Security theo mức baseline	PASS: Pod Security baseline được enforce mặc định
CIS 4.6.1 – Namespace Boundary	PASS: Audit quyền truy cập namespace và khắc phục thủ công nếu cần	PASS: Hệ thống enforce quyền truy cập chỉ trong namespace tương ứng
CIS 5.4.1 – Enable Flow Logs & Intra-node Visibility	PASS: Kích hoạt Flow Logs và Intra-node Visibility thông qua audit và cấu hình thủ công	PASS: Flow Logs và Intra-node Visibility được tự động bật

4.2 Tổng kết đồ án

Đồ án tập trung nghiên cứu và đánh giá mức độ tuân thủ CIS Google Kubernetes Engine (GKE) Autopilot Benchmark trong bối cảnh Kubernetes ngày càng được sử dụng rộng rãi trong kỷ nguyên điện toán đám mây. Thông qua việc triển khai và so sánh hai mô hình với 2 chế độ Standard và Autopilot, đồ án làm rõ vai trò của các nhóm khuyến nghị CIS Level 1, bao gồm **Policies** (RBAC, Pod Security Standards, Namespace) và **Managed Services** (Cluster Networking), nhằm giảm thiểu rủi ro bảo mật phát sinh từ cấu hình sai lệch và nâng cao mức độ an toàn cho hệ thống.

Về phương pháp thực hiện, nhóm áp dụng mô hình **Infrastructure as Code** với Terraform kết hợp Google Cloud SDK, Bash Script và GitHub Actions để tự động hóa quá trình triển khai, kiểm tra và khắc phục các cấu hình theo chuẩn CIS. Kết quả cho thấy cả hai mô hình đều có thể đáp ứng các yêu cầu bảo mật Level 1, tuy nhiên GKE Autopilot nổi bật với cơ chế bảo mật được enforced mặc định, giúp hạn chế đáng kể sai sót từ phía người dùng, trong khi GKE Standard mang lại tính linh hoạt cao nhưng đòi hỏi quy trình quản trị chặt chẽ hơn để đảm bảo tuân thủ tiêu chuẩn.

PHỤ LỤC

Bảng phân công:

STT	Họ và tên	Nhiệm vụ	Tỉ lệ %
1	Đinh Huỳnh Gia Bảo	- Tìm hiểu và soạn nội dung lý thuyết - Dùng Terraform để khởi tạo hạ tầng GKE Standard và Autopilot - Dùng Github Actions để cấu hình CI/CD - Dùng bash script để tự động hóa cho việc kiểm tra vai trò - Thiết kế slide thuyết trình và soạn nội dung báo cáo	Nghiêm túc thực hiện công việc và hoàn thành đúng tiến độ: 100%
2	Nguyễn Xuân Cường	- Tìm hiểu và soạn nội dung lý thuyết - Dùng bash script để tự động hóa cho PSS, ranh giới namespace và chính sách về mạng cho cụm GKE - Quay video demo - Thiết kế slide thuyết trình và soạn nội dung báo cáo	Nghiêm túc thực hiện công việc và hoàn thành đúng tiến độ: 100%

NHẬN XÉT CỦA GIẢNG VIÊN

[illegible]

TÀI LIỆU THAM KHẢO

- [1] Center for Internet Security, *CIS Google Kubernetes Engine (GKE) Autopilot Benchmark*, CIS SecureSuite, 2024. [Online]. Available: <https://www.cisecurity.org/cis-benchmarks>. [Accessed: 2025].
- [2] Center for Internet Security, *CIS Benchmarks December 2024 Update*, CIS, 2024. [Online]. Available: <https://www.cisecurity.org/insights/blog/cis-benchmarks-december-2024-update>. [Accessed: 2025].
- [3] Google Cloud, *CIS Benchmarks for Google Kubernetes Engine*, Google Cloud Documentation, 2024. [Online]. Available: <https://cloud.google.com/kubernetes-engine/docs/concepts/cis-benchmarks>. [Accessed: 2025].
- [4] Google Cloud, *GKE Autopilot Overview*, Google Cloud Documentation, 2024. [Online]. Available: <https://cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview>. [Accessed: 2025].
- [5] Google Cloud, *Security in GKE Autopilot*, Google Cloud Documentation, 2024. [Online]. Available: <https://cloud.google.com/kubernetes-engine/docs/concepts/security-overview>. [Accessed: 2025]

